FARMOON

Farm Produce Marketplace and Management System

A Project Report

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF COMPUTER APPLICATIONS

ST. JOSEPH'S ACADEMY OF
HIGHER EDUCATION AND RESEARCH

DEPARTMENT OF COMPUTER SCIENCE

BCA 2024-2028

INTRODUCTION

1. INTRODUCTION

FARMOON: The Farm Produce Marketplace and Management System is a web-based application developed to simplify and modernize the process of buying and selling farm produce. Traditionally, farmers face difficulty in reaching genuine buyers directly, negotiating fair prices, and managing orders, while customers struggle to find fresh produce from verified farmers without depending on middlemen. This project aims to address these challenges by providing a centralized digital platform that connects farmers directly with customers.

The system allows users (customers) to register, create profiles, browse farm produce listed by different farmers, add items to a cart, and place bookings for home delivery. Registered farmers can list their products along with photos, price, and stock details, manage their inventory, and track incoming bookings through the platform. Customers can view farmer profiles, rate and review farmers based on their experience, and make secure online payments for their orders.

An administrator oversees the entire system by managing districts, places, categories, sub-categories, user and farmer registrations, verifying farmer accounts, and monitoring orders and transactions across the platform. The platform also includes features such as complaint management, feedback submission, profile management, order and transaction tracking, and a crop recommendation tool for farmers, ensuring transparency and effective communication among all stakeholders.

A key highlight of FARMOON is that it goes beyond simply providing an online selling platform — it is designed to genuinely support farmers by helping them decide what to grow in the first place. The system integrates a Machine Learning based Crop Recommendation module, which uses a trained classification model to suggest the most suitable crop for a farmer's land based on soil nutrients (Nitrogen and Potassium levels), soil type, soil depth, temperature, humidity, and rainfall. By combining a direct-to-consumer selling platform with a data-driven crop recommendation system, FARMOON supports farmers at two critical stages of their livelihood — deciding what to cultivate, and then selling what they have cultivated at a fair price without middlemen.

Developed using the Django framework, FARMOON provides a secure, user-friendly, and efficient solution for direct farmer-to-consumer trade. By digitizing the produce-selling process, the system helps reduce dependency on middlemen, improves market access for farmers, ensures fair pricing, and promotes the sale of fresh, locally grown agricultural produce.

Objective

The main objective of FARMOON is to support farmers by providing a digital platform that simplifies and streamlines the process of buying and selling farm produce, connecting farmers directly with customers without the involvement of intermediaries, while also guiding farmers on what to cultivate through a Machine Learning based crop recommendation system.

Specific Objectives

➢ To provide an online platform for farm produce trading — enable farmers to list and sell their produce without depending on local middlemen or physically visiting markets.

➢ To connect farmers directly with customers — create a centralized system where customers can browse products from verified, registered farmers.

➢ To improve transparency in pricing — allow customers to view product details, prices, and farmer ratings before making a purchase decision.

➢ To reduce manual paperwork and effort — digitize product listing, booking, cart management, and payment processes, making trade faster and more efficient.

➢ To facilitate effective management — provide administrative tools for managing districts, places, categories, sub-categories, users, farmers, orders, and transactions.

➢ To enhance user experience — offer a user-friendly interface for browsing produce, managing the cart, tracking bookings, and communicating through complaints and feedback.

➢ To promote fair and sustainable agricultural trade — encourage farmers to reach a wider customer base and receive fair value for their produce.

➢ To ensure secure and organized data management — store user, farmer, product, booking, and payment information securely while maintaining data accuracy and accessibility.

➢ To assist farmers with decision-making — provide a crop recommendation feature and sales reports to help farmers plan production and track performance.

➢ To support farmers using Machine Learning — analyze soil nutrient levels (Nitrogen, Potassium), soil type, soil depth, temperature, humidity, and rainfall using a trained classification model, and recommend the most suitable crop for cultivation, thereby reducing crop failure and improving yield and income.

ABOUT THE ORGANIZATION

2. ABOUT THE ORGANIZATION

Progressive Software Solutions & Training is a fast developing, well established Information Technology enterprise with highly talented and efficient staff meeting the needs of clients throughout the world. The company is so popular that the clients feed the company with repeat orders, having faith in the capabilities. The staff includes management and engineering personnel, programmers, web developers, graphic designers, business analysts, technical writers etc. apart from the regular commercial and administrative hands. The motto of this reliable software provider is 'DO BEST; GET BEST'. Progressive Software Solutions & Training is committed to providing what they have promised. Progressive Software Solutions & Training strives for the satisfaction of their clients and dedicates themselves to better services for their customers.

Progressive Software Solutions & Training is famous for providing high quality, cost cutting, requirement oriented and value added web and software solutions with remarkable cyber services. Progressive Software Solutions & Training specializes in developing ever rewarding internet presence for ambitious companies which are in the hangover of swift moving internet facilities. Their aim is to grow with such organizations in a very fruitful manner. Their main concentration is to gather business oriented technological know-how and utilize it for the wellbeing of business concerns.

Progressive Software Solutions & Training experts in various technologies are committed to delivering effective and creative service to customers.

Services

- Software Development
- Web Development
- SEO Services (Search Engine Optimization)

SYSTEM ANALYSIS

3. SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

The existing system of farm produce trading is a traditional, manual, and unorganized approach. Here, farmers depend largely on local markets, wholesalers, and middlemen to sell their produce. Communication between farmers and buyers is limited, and farmers often have little control over the pricing of their own produce, since middlemen negotiate prices on their behalf. There is no centralized channel through which customers can directly discover and purchase fresh produce from nearby farmers. Farmers also have no organized way to track stock, bookings, or payments, and record-keeping is largely manual. Moreover, farmers generally decide what crop to cultivate based on tradition, guesswork, or informal advice from other farmers, rather than on any scientific analysis of their soil and climate conditions. This often leads to poor crop selection, reduced yield, and financial loss.

Disadvantages

- Time consuming and dependent on middlemen
- Farmers receive lower profit margins due to multiple intermediaries
- No direct communication channel between farmers and customers
- Difficult to track stock, orders, and payments manually
- Lack of transparency in pricing and quality information
- No scientific or data-driven guidance for farmers to choose the right crop for their soil and climate, leading to crop failure and reduced income

3.2 PROPOSED SYSTEM

The proposed system, FARMOON, computerizes the entire farm produce trading process and helps farmers interact directly with customers. It is a comprehensive, integrated web platform designed to enhance market access for farmers, ensure fair pricing, and provide a convenient online shopping experience for customers. It allows farmers to list their produce with photographs, pricing, and stock details, and allows customers to browse, add items to a cart, and place bookings that are delivered directly to their address. The system also provides administrative oversight for verifying farmers, managing categories and locations, and monitoring orders and transactions. Compared to the existing manual system, FARMOON offers greater transparency, faster transactions, and improved security and privacy of data.

In addition to enabling direct selling, FARMOON proposes a Machine Learning based Crop Recommendation module to genuinely support farmers before the selling stage even begins. A farmer can enter soil and climate parameters — Nitrogen level, Potassium level, soil type, soil depth, temperature, humidity, and rainfall — and the system uses a pre-trained K-Nearest Neighbors (KNN) classification model to predict and recommend the most suitable crop for those conditions. This reduces reliance on guesswork, helps farmers make informed cultivation decisions, and ultimately supports better yield and income, which they can then sell directly to customers through the same platform.

3.3 SOFTWARE REQUIREMENTS SPECIFICATION

Hardware Specification

The use of sufficient hardware is very important for the proper working of any software. Following are the hardware required by the system.

Component	Requirement
Processor	Intel Core i5
Processor Speed	1.5 GHz or higher
System Bus	64 Bit
Memory	4 GB RAM or higher
Hard Disk	40 GB or higher
Monitor	15.6" LCD Monitor
Keyboard	104 Keys

Software Specification

Software is a part of a computer system that consists of encoded information or computer instructions, in contrast to the physical hardware from which the system is built. Following are the software required by the system.

Component	Requirement
Operating System	Windows 11
IDE	Visual Studio Code
Front End	PYTHON DJANGO, HTML5, CSS, JavaScript
Back End	SQLite3
Machine Learning	Python, Scikit-learn (KNN Classifier), Pandas, Joblib
Web Server	Django Development Server / Apache
Browser	Google Chrome, Mozilla Firefox, Microsoft Edge

3.4 FEASIBILITY STUDY

The main objective of this study is to determine whether the proposed system is feasible. Mainly there are three types of feasibility study to which the proposed system is subjected, as described below:

- Economic feasibility
- Technical feasibility
- Behavioral feasibility

The proposed system must be evaluated from a technical viewpoint first, and if technically feasible then its impact on the organization must be assessed. If compatible, the behavioural feasibility can be devised. Then it must be tested for economic feasibility.

Economic Feasibility

Economic analysis is used for evaluating the cost effectiveness of a proposed system. Economically speaking, our application is never cost related. It is easy to implement, and the user can conveniently use our application without any kind of exceptions or errors. Our application, being simple to interact with, is economically feasible.

Technical Feasibility

The technical requirements of our system are highly affordable. There is no difficulty in migrating from the existing system to our proposed system. It is platform independent, and the application will run smoothly on any kernel without causing any trouble to the end user.

Behavioral Feasibility

The system is intended for personal and commercial use by farmers and customers. Since the software is simple and provides a good Graphical User Interface, it provides a high level of user acceptance.

3.5 DATA FLOW DIAGRAM

To start the system design, something analogous to an architectural blueprint is required as a starting point for design. It is a way to focus on functions rather than physical implementation. One such tool is a DFD.

Structured analysis is a set of techniques and graphical tools that help the analyst develop a new kind of system specification that is easily understandable to the user. DFDs show the major decompositions of the system functions and their interfaces. The DFD is graphic and presents a picture of what is being specified, and is conceptually easy to understand and present the application.

One important feature of DFDs is that they are logical rather than physical. The elements of the system do not depend on vendor or hardware. They specify, in a precise, concise manner, the working of the system and how it hangs together.

DFD is the graphic representation of data movement, processes, and files used in support of an information system. There are several rules of thumb used in drawing DFDs.

- Process should be named and numbered for easy reference.
- The direction of flow is from top to bottom and from left to right.
- When a process is expanded into lower level details, they must be numbered.
- Process and data flow names must have the first letter of each word capitalized.

Context Level Diagram

The context level DFD represents the overall system as a single process interacting with the three external entities of FARMOON — Admin, Farmer, and User — through requests and responses.

External Entity	Sends to FARMOON	Receives from FARMOON
Admin	Verification & management requests	Confirmations & reports
Farmer	Product, stock & booking requests	Order details & responses
User	Registration, booking & payment requests	Product details & order status

Level 1 DFD for Admin

The Admin interacts with the system through the following processes, each linked to its corresponding data store:

Process	**Data Store**
Login	tbl_adminregistration
District Management	tbl_district
Place Management	tbl_place
Category Management	tbl_category
Subcategory Management	tbl_subcategory
User Verification	tbl_userregistration
Farmer Verification	tbl_farmerregistration
Order Management	tbl_booking, tbl_cart
Transaction Management	tbl_payment
Complaint & Feedback Management	tbl_complaint, tbl_feedback

Level 1 DFD for Farmer

Process	**Data Store**
Registration	tbl_farmerregistration
Login	tbl_farmerregistration
Profile Management	tbl_farmerregistration
Add / Manage Product	tbl_product

Stock Management	tbl_stock
Gallery Management	tbl_gallery
View Booking / Order Processing	tbl_booking, tbl_cart
View Rating	tbl_rating
Crop Recommendation (ML Model)	knn_model.pkl (Trained ML Model)
Sales Report	tbl_booking, tbl_payment

3.6 MACHINE LEARNING BASED CROP RECOMMENDATION

FARMOON supports farmers not only by helping them sell their produce but also by helping them decide what to grow, through an integrated Machine Learning based Crop Recommendation system. This module is built using Python's Scikit-learn library and is loaded into the Django application using Joblib.

The recommendation model is a K-Nearest Neighbors (KNN) classifier that has been pre-trained on agricultural data relating soil and climatic parameters to suitable crops. When a farmer wants a recommendation, the system collects the following input parameters through a simple web form:

Input Parameter	**Description**
Nitrogen (N)	Nitrogen content present in the soil
Potassium (K)	Potassium content present in the soil
Temperature	Average temperature of the region (°C)
Humidity	Relative humidity of the region (%)
Rainfall	Average rainfall of the region (mm)
Soil Type	Category of soil — Alluvial, Black, Clayey, Loamy, or Sandy
Soil Depth	Depth of the cultivable soil layer (cm)

These inputs are organized into a structured data record and passed to the trained KNN model, which compares the input against patterns learned from historical crop data and predicts the crop that is most likely to grow successfully under the given soil and climatic conditions. The predicted crop name is then displayed to the farmer on the Crop Recommendation page, enabling the farmer to make an informed, data-driven cultivation decision instead of relying purely on tradition or guesswork.

By pairing this recommendation system with the produce-selling platform, FARMOON supports the farmer across the complete cycle — from deciding what to cultivate, to growing it, to finally selling it directly to customers at a fair price.

Level 1 DFD for User

Process	**Data Store**
Registration	tbl_userregistration
Login	tbl_userregistration
Profile Management	tbl_userregistration
View Farmers / Products	tbl_farmerregistration, tbl_product
Cart Management	tbl_cart
Booking Management	tbl_booking
Payment	tbl_payment
Rating & Review	tbl_rating
Complaint Management	tbl_complaint
Feedback Management	tbl_feedback

SYSTEM DESIGN

4. SYSTEM DESIGN

System design is the creative act of developing new inputs, a database, offline files, methods, procedures, and outputs for processing business data to meet organizational objectives. The design phase focuses on the detailed implementation of the system recommended in the feasibility study. The design phase is a transition from a user-oriented document to a document oriented towards the programmers or database personnel. Characteristics of a well-designed system are:

- Accessibility
- Decision making ability
- Economy
- Flexibility
- Reliability
- Simplicity

The design will determine the success of the system. System design is based on the information gathered during system analysis. System design goes through two phases of development.

- Logical design - DFD shows the logical flow of a system and defines the boundaries of the system. For the candidate system, it describes the inputs, outputs, databases and procedures — all in a format that meets the user's requirements.
- Physical design - This produces the working system by defining the design specification that tells programmers exactly what the candidate system must do.

4.1 INPUT DESIGN

Input design is the link that ties the information system into the world of its users. The input design involves determining what the inputs are, how the data should be entered, how to validate data, how to minimize data entry, and how to provide a multi-user facility accurately.

Input data are the most common cause of error in data processing. Errors entered by data entry operators can be controlled by input design. Input design is the process of converting user-originated inputs to a computerbased format. Input data are collected and organized into groups of similar data. Once identified, appropriate input media are selected for processing. All the input data are validated in order, and if any data violates a condition, the user is warned by a message. If the data satisfies all the conditions, then it is transferred to the appropriate tables in the database.

In FARMOON, the input design covers forms such as User Registration, Farmer Registration, Product Addition, Stock Update, Cart Addition, Booking, Payment, Complaint, Feedback, and Rating, each with proper client-side and server-side validation.

4.2 OUTPUT DESIGN

Output design involves specifying how production of on-screen reports and page based reports will occur. Output may occur to a database or file for storing information entered, or also for use by other systems. Computer output is the most important and direct source of information to the user. Output design is a very important phase because the output needs to be presented in an attractive manner. Efficient, intelligible output design improves the system's relationship with the user and helps in decision making. A major form of the output is the hard copy from the printer and screen reports. Printouts are designed around the output requirements of the user. Allowing the user to view a sample screen is important because the user is the ultimate judge of the quality of output. The output model of this system is the user-friendly window. These user-friendly windows are meant for the purpose of easy viewing of the stored information.

In FARMOON, output design includes the product listing pages, farmer dashboards with stock and sales summaries, order and transaction reports for the admin, invoices/booking confirmations for users, and exportable order and transaction lists.

4.3 TABLE DESIGN

A database is a collection of data. Database design refers to the design of the tables used to store data. The database involves the names of records, data items with their name, type and size. In the design of the database, first we must thoroughly look into the requirements of the program for the design of the database. Then we must design how many tables are required in the database. Thereafter, as per the requirement of the end users, we can decide which fields must be in each table. As a general rule, a provision must be taken in the design for the future enhancement of the program. Some of the important tables are:

1. Table name: tbl_district

Table description: To store the details of the district

Field Name	Data Type	Constraints	Field Description
id	int	Primary key	District id
district_name	varchar(100)	Not null	Name of the district

2. Table name: tbl_place

Table description: To store the details of place

Field Name	Data Type	Constraints	Field Description
id	int	Primary key	Place id
place_name	varchar(30)	Not null	Name of the place
district	int	Foreign key (tbl_district)	District id

3. Table name: tbl_category

Table description: To store the details of the product category

Field Name	Data Type	Constraints	Field Description
id	int	Primary key	Category id
category_name	varchar(30)	Not null	Name of category

4. Table name: tbl_subcategory

Table description: To store the details of the product sub-category

Field Name	Data Type	Constraints	Field Description
id	int	Primary key	Subcategory id
subcategory_name	varchar(30)	Not null	Name of subcategory
category	int	Foreign key (tbl_category)	Category id

5. Table name: tbl_adminregistration

Table description: To store the details of the admin

Field Name	Data Type	Constraints	Field Description
id	int	Primary key	Admin id
adminregistration_name	varchar(50)	Not null	Name of admin
adminregistration_email	varchar(50)	Not null	Email of admin
adminregistration_password	varchar(50)	Not null	Password of admin

6. Table name: tbl_userregistration

Table description: To store the details of the customer / user

Field Name	Data Type	Constraints	Field Description
id	int	Primary key	User id
userregistration_name	varchar(100)	Not null	Name of user
userregistration_email	varchar(100)	Not null	Email of user
userregistration_contact	varchar(100)	Not null	Contact of user
userregistration_address	varchar(100)	Not null	Address of user
userregistration_gender	varchar(100)	Not null	Gender of user
userregistration_dateofbirth	date	Not null	Date of birth of user
userregistration_place	int	Foreign key (tbl_place)	Place id
userregistration_photo	file	Not null	Photo of user
userregistration_password	varchar(100)	Not null	Password of user
user_status	int	Default 0	Status of user (Pending/Accepted/Rejected)

7. Table name: tbl_farmerregistration

Table description: To store the details of the farmer

Field Name	Data Type	Constraints	Field Description
id	int	Primary key	Farmer id
farmer_name	varchar(100)	Not null	Name of farmer
farmer_email	varchar(100)	Not null	Email of farmer
farmer_contact	varchar(100)	Not null	Contact of farmer
farmer_address	varchar(100)	Not null	Address of farmer
farmer_photo	file	Not null	Photo of farmer
farmer_proof	file	Not null	Identity proof of farmer
farmer_password	varchar(100)	Not null	Password of farmer
farmer_status	int	Default 0	Status of farmer (Pending/Accepted/Rejected)
place_id	int	Foreign key (tbl_place)	Place id

8. Table name: tbl_product

Table description: To store the details of farm produce listed by farmers

Field Name	Data Type	Constraints	Field Description
id	int	Primary key	Product id
product_name	varchar(100)	Not null	Name of product
product_details	varchar(100)	Not null	Description of product
product_price	varchar(100)	Not null	Price of product
product_photo	file	Not null	Photo of product
category_id	int	Foreign key (tbl_category)	Category id
farmer_id	int	Foreign key (tbl_farmerregistration)	Farmer id

9. Table name: tbl_stock

Table description: To store stock quantity details of products

Field Name	Data Type	Constraints	Field Description
id	int	Primary key	Stock id
stock_quantity	varchar(100)	Not null	Quantity in stock
stock_date	date	Not null	Date of stock update
product_id	int	Foreign key (tbl_product)	Product id

10. Table name: tbl_gallery

Table description: To store additional images of a product

Field Name	Data Type	Constraints	Field Description
id	int	Primary key	Gallery id
gallery_photo	file	Not null	Additional photo of product
product_id	int	Foreign key (tbl_product)	Product id

11. Table name: tbl_booking

Table description: To store booking / order details placed by users

Field Name	Data Type	Constraints	Field Description
id	int	Primary key	Booking id
booking_date	date	Not null	Date of booking
booking_time	time	Not null	Time of booking
booking_status	int	Default 0	Status of booking
booking_amount	varchar(60)	Not null	Total amount of booking
userid	int	Foreign key (tbl_userregistration)	User id
delivery_address	text	Null allowed	Delivery address of the order

12. Table name: tbl_cart

Table description: To store cart items linked to a booking

Field Name	Data Type	Constraints	Field Description
id	int	Primary key	Cart id
cart_quantity	int	Default 1	Quantity of product in cart
cart_status	int	Default 0	Status of cart item
productid	int	Foreign key (tbl_product)	Product id
bookingid	int	Foreign key (tbl_booking)	Booking id

13. Table name: tbl_payment

Table description: To store payment details of a booking

Field Name	Data Type	Constraints	Field Description
id	int	Primary key	Payment id
bookingid	int	Foreign key (tbl_booking)	Booking id
payment_amount	decimal(10,2)	Not null	Amount paid
payment_method	varchar(50)	Default 'Online Payment'	Method of payment
payment_status	varchar(20)	Default 'Success'	Status of payment
payment_date	datetime	Not null	Date & time of payment

14. Table name: tbl_rating

Table description: To store user ratings and reviews given to farmers

Field Name	Data Type	Constraints	Field Description
id	int	Primary key	Rating id
rating_data	int	Not null	Rating value
user	int	Foreign key (tbl_userregistration)	User id
user_review	varchar(500)	Not null	Review content
farmer	int	Foreign key (tbl_farmerregistration)	Farmer id
datetime	datetime	Not null	Date & time of rating

15. Table name: tbl_complaint

Table description: To store complaints raised by users

Field Name	Data Type	Constraints	Field Description
id	int	Primary key	Complaint id
complaint_title	varchar(100)	Not null	Title of the complaint
complaint_content	varchar(100)	Not null	Content of the complaint
complaint_date	date	Not null	Date of the complaint
complaint_status	int	Default 0	Status of the complaint
complaint_reply	varchar(100)	Null allowed	Admin's reply to complaint
user	int	Foreign key (tbl_userregistration)	User id

16. Table name: tbl_feedback

Table description: To store feedback given by users about the platform

Field Name	Data Type	Constraints	Field Description
id	int	Primary key	Feedback id
feedback_content	varchar(100)	Not null	Content of feedback
feedback_date	date	Not null	Date of the feedback
feedback_rating	int	1 to 5	Star rating of feedback
user	int	Foreign key (tbl_userregistration)	User id

Crop Recommendation Model Artifact

Unlike the other modules, the Crop Recommendation feature does not store its knowledge in a database table.

Instead, a pre-trained K-Nearest Neighbors (KNN) classification model is serialized and stored as a model file (knn_model.pkl) inside the project's Assets/Model directory. This file is loaded once into memory using Joblib when the Farmer application starts, and is then used to generate crop predictions on demand from the soil and climate values submitted by the farmer, without requiring a separate database table.

4.4 PROCESS DESIGN

There are 3 modules in the system.

1. Admin

- Login
- District Management
- Place Management
- Category Management
- Subcategory Management
- User Verification
- Farmer Verification
- Order Management
- Transaction Management
- Complaint & Feedback Management

2. Farmer

- Registration
- Login
- Profile Management
- Add / Manage Product
- Stock Management
- Gallery Management
- View Booking / Order Processing
- View Rating
- Crop Recommendation
- Sales Report

3. User

- Registration
- Login
- Profile Management
- View Farmers / Products
- Cart Management
- Booking Management
- Payment
- Rating & Review
- Complaint Management
- Feedback Management

SYSTEM TESTING AND IMPLEMENTATION

5. SYSTEM TESTING & IMPLEMENTATION

Testing is the process of examining the software to compare the actual behavior of the system with the expected behavior. The major objective of software testing is to ensure that faults and errors are identified and eliminated before the system is deployed. To achieve this goal, the tester executes the program with the intention of finding defects and verifying that the software functions correctly. Although testing cannot guarantee the complete absence of errors, it helps to ensure that the system operates reliably and efficiently.

System testing is defined as the process of detecting defects in software and verifying that the developed system satisfies the specified requirements. It is one of the most important phases in software development because it provides the final opportunity to identify and rectify errors that may have occurred during the design and coding stages. Testing is carried out to ensure the correctness, reliability, maintainability, and performance of FARMOON.

FARMOON was developed to provide an efficient platform for farmers to sell produce and for customers to purchase farm produce online. The system allows users to register, browse products, add items to a cart, place bookings, make payments, and communicate through complaints and feedback. Farmers can list products, manage stock, view bookings, and generate sales reports. Administrators can manage locations, categories, users, and farmers effectively. Therefore, thorough testing was required to ensure that all modules functioned accurately and efficiently.

System testing is the first stage of implementation and is aimed at ensuring that the developed system works correctly before it is made available for actual use. Testing plays a vital role in the success of the system. It is based on the assumption that if all individual modules perform correctly and are properly integrated, then the overall objectives of the system will be achieved successfully. A series of tests were conducted on the proposed system before it was made ready for user acceptance testing.

The testing steps carried out for FARMOON are:

- Unit Testing
- Integration Testing
- Validation Testing
- Output Testing
- Acceptance Testing

System testing provides assurance that the software, once validated, can be integrated with all other system elements successfully. It verifies whether all modules have been combined properly and whether the overall system functionality and performance requirements have been achieved. After integrating the User Module, Farmer Module, and Admin Module, validation testing was performed on the complete system. The results showed that all modules worked together effectively and satisfied the overall objectives of FARMOON.

Unit Testing

Unit testing is carried out module-wise, with each module being treated as an independent unit. Attention is focused on individual modules separately in order to identify coding errors, logical errors, and databaserelated issues. This approach helps ensure that each module performs its intended function correctly before integration with other modules.

Various test cases were prepared and executed for each module of FARMOON. The results obtained were compared with the expected outputs to verify that the modules functioned according to the specified requirements.

The following modules were subjected to unit testing:

- User Registration Module
- User Login Module
- Farmer Registration Module
- Product Addition & Stock Module
- Cart & Booking Module
- Payment Module
- Rating & Review Module
- Complaint Management Module
- Feedback Management Module
- Admin Management Module
- Machine Learning Crop Recommendation ModuleDuring unit testing:
- Module interfaces were tested to ensure that information flowed correctly between forms, modules, and the database.
- Boundary conditions were tested to ensure that the modules operated properly within the specified limits and restrictions.
- All independent execution paths through the control structures were executed to ensure that every statement in the module was executed at least once.
- Input validation was tested to ensure that invalid or incomplete data was rejected appropriately.
- Database operations such as insertion, updation, deletion, and retrieval were verified.
- Error handling paths were tested to ensure that the system responded correctly to unexpected or invalid inputs.

For the Machine Learning Crop Recommendation module, unit testing additionally involved verifying that the KNN model file loaded correctly using Joblib, that all seven input parameters (Nitrogen, Potassium, temperature, humidity, rainfall, soil type, and soil depth) were accepted and correctly converted to numeric form, that missing or invalid inputs were rejected by the form validation, and that the model consistently returned a valid crop prediction for a given set of inputs.

The results of unit testing showed that all modules performed according to the specified requirements and produced the expected outputs.

Integration Testing

Integration testing is a systematic technique for constructing the program structure while simultaneously conducting tests to uncover errors associated with interfacing between modules. The individually unit-tested modules were integrated to form a complete system structure as specified in the design phase. Incremental integration testing was adopted in this project.

The purpose of integration testing is to verify the communication and interaction between different modules of FARMOON. After ensuring that each module functioned correctly in isolation, the modules were integrated and tested together to identify any interface errors, data transfer issues, or functional inconsistencies.

The following integrations were tested:

- User Registration and Login Module
- User Module and Product Browsing Module

- Cart Module and Booking Module
- Booking Module and Payment Module
- User Module and Complaint Management Module
- User Module and Feedback Management Module
- Farmer Module and Product / Stock Management Module
- Farmer Module and Booking Processing Module
- Farmer Module and Machine Learning Crop Recommendation Module
- Admin Module and User Verification Module
- Admin Module and Farmer Verification ModuleDuring integration testing:
- Data flow between modules was verified to ensure that information was transferred accurately.
- Database interactions between integrated modules were tested to ensure consistency and integrity of stored data.
- Interfaces between User, Farmer, and Admin modules were tested thoroughly.
- Communication between modules was checked to ensure that all integrated functions operated according to system requirements.
- Error conditions occurring during module interaction were identified and corrected.

The integration testing process confirmed that all modules worked together successfully. FARMOON performed as an integrated application without any significant interface errors, and all system functionalities operated according to the specified requirements.

Validation Testing

Validation testing is performed to ensure that the developed software satisfies all user requirements and functions according to the specified objectives. It is carried out after integration testing to verify that the complete system behaves as expected in a real-world environment.

The users and farmers conduct testing on the system and provide feedback regarding its functionality and usability. The developer is not directly involved during this testing phase, making it possible to evaluate the software under actual operating conditions. Any issues identified during testing are reported, analyzed, and corrected through suitable modifications to the system.

In FARMOON, client-side and server-side validation techniques are implemented to ensure that only valid data is accepted by the system. Validation testing was performed on various input fields such as:

- User / Farmer Name
- Email Address
- Mobile Number
- Password
- Product Details and Price
- Stock Quantity
- Booking / Delivery Address
- Payment Amount
- Crop Recommendation Inputs (Nitrogen, Potassium, Temperature, Humidity, Rainfall, Soil Type, Soil Depth)

Incorrect or invalid values entered in these fields are detected and rejected by the system. Appropriate error messages are displayed to guide the user in providing valid information. For example, invalid email formats, incorrect mobile numbers, empty mandatory fields, and invalid passwords are not accepted by the system.

Validation testing also ensured that:

- All mandatory fields were properly validated.
- Invalid data entries were rejected.
- Appropriate error messages were displayed for incorrect inputs.
- User and farmer registration and login processes accepted only valid credentials.
- Product listing, cart, and booking submissions were validated before storage in the database.

The results of validation testing confirmed that FARMOON correctly validates user inputs and prevents invalid data from entering the database, thereby improving system reliability and data integrity.

Output Testing

Output testing is performed to ensure that FARMOON generates accurate and user-friendly outputs. The outputs displayed on the screen were verified against user requirements and expected results. The details related to product listings, bookings, payments, complaints, feedback, and verification processes were tested and found to be correct.

The output format was clear, well-organized, and easy for users to understand. Since all outputs were generated accurately and satisfied user requirements, no major modifications were required after output testing.

Acceptance Testing

Acceptance testing is performed on the completed system to ensure that it satisfies user requirements and operates correctly in the intended environment. Each test case is executed to verify the functionality and usability of the system, resulting in either a pass or fail outcome.

Acceptance testing is carried out by end users to determine whether FARMOON meets their expectations. The system was developed according to user requirements, and the interfaces were designed to be simple, clear, and user-friendly.

The users successfully performed activities such as registration, browsing products, adding items to the cart, placing bookings, making payments, and submitting complaints and feedback without any difficulties. The system passed the acceptance testing phase successfully and was accepted by the users, as it fulfilled all specified requirements.

Debugging

Debugging is the process of identifying and removing errors or bugs from software to ensure its proper functioning. It helps in detecting coding errors, logical mistakes, and runtime issues that may affect system performance.

During the development of FARMOON, debugging was carried out to identify and correct errors in user and farmer registration, login, product and stock management, cart and booking management, payment processing, and database operations. Various debugging techniques and tools were used to locate the source of errors and resolve them effectively.

The debugging process involved:

- Reproducing the problem.

- Identifying and analyzing the cause of the error.
- Examining the program state and variable values.
- Correcting the identified error.
- Retesting the system to ensure that no new errors were introduced.

Through proper debugging, the system was made more reliable, stable, and efficient, ensuring smooth operation of all modules.

Implementation

Implementation is the final and most important phase of system development. It involves converting the designed system into a working application through coding, testing, user training, and deployment. During this phase, FARMOON takes its actual form and becomes ready for practical use.

The implementation phase is concerned with translating the system design into source code using Python Django, HTML, CSS, JavaScript, and the SQLite database. Before deployment, extensive testing was carried out to ensure that the system operated according to the specified requirements without any major errors.

The developed system was tested by users, and necessary modifications were made based on their feedback. After successful testing and validation, the system was implemented and made operational. The implementation process ensured a smooth transition from the existing manual process to the computerized FARMOON system.

The implementation stage involved the following activities:

- Testing the developed system.
- Detection and correction of errors.
- User training and guidance.
- Database setup and configuration.
- System deployment and operation.
- Evaluation of system performance.

FARMOON was successfully implemented and found to be efficient, reliable, and user-friendly. The system fulfilled all specified requirements and performed all operations accurately.

Implementation Plans

- ➢ Testing the system with sample user, farmer, product, and booking data.
- ➢ Detection and correction of errors identified during testing.
- ➢ Making necessary modifications and improvements to the system.
- ➢ Comparing the developed system with the existing manual process to verify efficiency and accuracy.
- ➢ Installation and configuration of required software utilities, including Python Django, SQLite Database, and supporting tools.
- ➢ Training and guidance for administrators, users, and farmers regarding system operation.
- ➢ Database setup and deployment of FARMOON.
- ➢ Monitoring and evaluation of system performance after implementation.

SECURITY TECHNOLOGIES AND POLICIES

6. SECURITY TECHNOLOGIES AND POLICIES

System Security

Threats to System Security

Any act, event, or circumstance that poses a risk to the system and its data is known as a threat. Management must be aware of various security threats in order to protect system resources through proper policies, security measures, and technology.

A threat may be intentional, such as unauthorized access by hackers, or accidental, such as hardware failures and natural disasters. FARMOON stores important information related to users, farmers, products, bookings, payments, complaints, and feedback. Therefore, adequate security measures are necessary to protect the system from potential threats.

The major threats to system security include:

- ➢ Unauthorized access to the system and modification or deletion of stored data.
- ➢ Attempts to gain access to user or farmer accounts through password attacks.
- ➢ Uploading malicious software such as viruses, worms, and Trojan programs.
- ➢ Hardware failures that may result in data loss.
- ➢ Power fluctuations and excess electrical supply affecting system operation.
- ➢ Natural disasters such as floods, earthquakes, and fire accidents.
- ➢ Database corruption or accidental deletion of records.
- ➢ Network-related attacks that may affect system availability and performance.

Proper security mechanisms, user authentication, data backup procedures, and access control techniques are implemented to minimize the impact of these threats and ensure the safe operation of FARMOON.

Security Technologies and Policies

Security and maintenance are important aspects of any software system. Maintenance involves modifying and updating the software to ensure that it continues to function efficiently and meets changing user requirements. It includes correcting errors, improving performance, and making necessary enhancements to the system after deployment.

FARMOON requires regular maintenance to ensure smooth operation and reliability. Maintenance activities include fixing reported issues, updating system features, improving user interfaces, and adapting the software to changes in hardware and software environments.

Since the system stores important information such as user and farmer details, product information, bookings, payments, complaints, and feedback, adequate security measures are essential. Security mechanisms are implemented to protect the system from unauthorized access, data loss, and malicious activities.

The following security measures are incorporated in the system:

- ➢ User authentication through valid username and password.
- ➢ Access control based on user roles such as Administrator, Farmer, and User.
- ➢ Protection of sensitive data stored in the database.
- ➢ Validation of user inputs to prevent invalid or malicious data entry.
- ➢ Secure management of product, booking, and payment records.
- ➢ Backup and recovery mechanisms to prevent data loss.
- ➢ Monitoring and maintenance of system performance and security.

The system allows access only to authorized users with valid login credentials. These security technologies and policies help ensure the confidentiality, integrity, and availability of information within FARMOON.

System Security Measures

Security is an important factor in FARMOON. Since the system stores user and farmer details, product information, bookings, payments, complaints, and feedback, appropriate security measures are implemented to prevent unauthorized access and protect confidential data.

1. Passwords

The most common and effective method of security is password protection. A password is a secret combination of characters used for user authentication and access control. It verifies the identity of users before granting access to the system.

In FARMOON, every Administrator, Farmer, and User must log in using a valid email and password. This prevents unauthorized users from accessing sensitive information and performing restricted operations.

2. Role-Based Access Control

During login, the system identifies whether the credentials belong to a User, Farmer, or Admin, and redirects each to their respective dashboard, ensuring that each role can access only the modules and data relevant to it. Newly registered users and farmers remain in a 'Pending' state until verified and accepted by the Administrator, and rejected accounts are denied access.

3. Data Security

All important information related to users, farmers, products, bookings, payments, ratings, complaints, and feedback is stored securely in the database. Access to data is restricted based on user roles and permissions.

4. Input Validation

The system validates all user inputs before storing them in the database. Invalid, incomplete, or malicious data entries are rejected, ensuring data integrity and system reliability.

These security measures help maintain the confidentiality, integrity, and availability of information within FARMOON.

MAINTENANCE

7. MAINTENANCE

Maintenance is the process of modifying a software system after its deployment in order to correct faults, improve performance, adapt to changing requirements, and enhance system functionality. Maintenance is an important phase of the software development life cycle because it ensures that the system continues to operate efficiently and effectively over time.

In FARMOON, maintenance includes correcting coding and design errors, updating system documentation, improving user interfaces, and enhancing existing features. It also involves updating the system according to user feedback and changing technological requirements.

Maintenance activities help to keep the system reliable, secure, and user-friendly. Any issues reported by users or farmers can be identified and corrected promptly. The system can also be upgraded by adding new features and improving existing functionalities whenever required.

The future updates and maintenance can be carried out even after the system has been delivered to users. Maintenance is not limited to bug fixing alone; it also includes adapting the software to changing environments, improving performance, and meeting new user requirements.

Software maintenance is a broad activity that includes error correction, capability enhancement, optimization, and system improvement. Since changes are inevitable, appropriate mechanisms must be adopted for evaluating, controlling, and implementing modifications. The primary objective of maintenance is to preserve and enhance the value of the software over time.

The categories of maintenance are:

Corrective Maintenance

Corrective maintenance involves identifying and fixing errors, bugs, and defects discovered after the system is deployed. It ensures the smooth functioning of FARMOON.

Adaptive Maintenance

Adaptive maintenance is performed to modify the system according to changes in the operating environment, technology, hardware, software, or user requirements.

Perfective Maintenance

Perfective maintenance focuses on improving system performance, usability, efficiency, and adding new features based on user feedback and future requirements.

Corrective Maintenance

Corrective maintenance refers to the modifications made to FARMOON to correct defects and errors discovered after deployment. These defects may arise from design errors, logical errors, coding mistakes, database inconsistencies, or improper handling of user inputs.

Design errors may occur when system requirements are misunderstood or implemented incorrectly. Logic errors may result from incorrect conditions, calculations, or program flow. Coding errors are caused by mistakes during the implementation of program logic and database operations. In addition, data processing errors and performance issues may also require corrective maintenance.

When a system failure or unexpected behavior is identified, corrective actions are taken to restore normal system operation. The errors are analyzed, corrected, and thoroughly tested before being applied to the live system. This helps maintain the reliability and efficiency of FARMOON.

Proper corrective maintenance ensures that issues related to user and farmer registration, product and stock management, cart and booking, payment processing, complaints, and feedback handling are resolved quickly and effectively. It also minimizes the impact of errors on users and improves overall system performance.

Adaptive Maintenance

Adaptive maintenance involves modifying FARMOON to accommodate changes in its operating environment. The environment includes hardware platforms, software technologies, databases, operating systems, security requirements, and organizational policies that may change over time.

As technology evolves, the system may require updates to remain compatible with newer versions of software and hardware. Adaptive maintenance ensures that the application continues to function efficiently under these changing conditions.

This type of maintenance may include upgrading the Django framework, updating database configurations, migrating the system to a new server, enhancing authentication mechanisms, or adapting the application to new operating systems and web browsers. Changes in government regulations related to agricultural trade, food safety, or data security requirements may also require modifications to the system.

Adaptive maintenance helps FARMOON remain reliable, compatible, and effective in a continuously changing technological and operational environment.

Perfective Maintenance

Perfective maintenance refers to modifications made to improve the performance, efficiency, usability, and functionality of FARMOON. As users continue to use the system, new requirements and suggestions may arise that help enhance the overall quality of the application.

This type of maintenance focuses on improving existing features and adding new functionalities to meet changing user needs. Enhancements may include improving the user interface, optimizing database operations, increasing system performance, generating additional reports, and introducing advanced features such as real-time notifications, online payment gateway integration, or mobile application support.

Perfective maintenance also aims to make the software more efficient, user-friendly, well-documented, and easier to maintain. By continuously improving the system, FARMOON can provide better services to users, farmers, and administrators while adapting to future requirements and technological advancements.

CONCLUSION

8. CONCLUSION

Scope for Future Enhancement

FARMOON has been successfully developed and implemented according to the requirements specified by the users. The system has been thoroughly tested and found to be functioning correctly without major errors. It provides an efficient platform for connecting farmers directly with customers, including farmer and user registration, product and stock management, cart and booking management, payment processing, rating and review, complaint management, feedback management, and administrative controls.

The system reduces dependency on middlemen, improves data accuracy, and provides better communication between users, farmers, and administrators. Through proper authentication and secure database management, the system ensures reliable and secure operation.

Although the current system satisfies all major requirements, future enhancements can further improve its functionality and efficiency. Some possible future enhancements include:

- Integration of a real online payment gateway for secure transactions.
- Mobile application development for Android and iOS platforms.
- Real-time notifications through SMS and email services.
- GPS-based farmer and delivery location tracking.
- Improving the Machine Learning crop recommendation model by training it on a larger and more region-specific agricultural dataset for higher accuracy.
- Extending the ML system with market price prediction, so farmers can also estimate the expected selling price of a recommended crop.
- Automatically fetching live temperature, humidity, and rainfall values from a weather API instead of manual entry, to make crop recommendation more accurate and convenient.
- Integration with government agricultural databases and schemes.
- Advanced reporting and analytics for administrators and farmers.
- Cloud-based deployment for improved scalability and accessibility.
- Real-time tracking of order and delivery status.
- Enhanced security features using advanced authentication mechanisms such as OTP verification.

The future updates and maintenance can be performed even after the system has been delivered to users. By modifying existing components and adding new functionalities, the system can continue to evolve according to changing user requirements and technological advancements. Maintenance not only includes fixing errors but also adapting the software to new environments, improving performance, and enhancing overall user experience.

Thus, FARMOON provides a strong foundation for efficient, direct farmer-to-consumer trade and has significant potential for future growth and enhancement.

BIBLIOGRAPHY

9. BIBLIOGRAPHY

[1] Pressman, Roger S. Software Engineering: A Practitioner's Approach, McGraw-Hill Education, 8th Edition, 2014.

[2] Silberschatz, Abraham, Korth, Henry F., and Sudarshan, S. Database System Concepts, McGraw-Hill Education, 6th Edition, 2011.

[3] Elmasri, Ramez and Navathe, Shamkant B. Fundamentals of Database Systems, Pearson Education, 7th Edition, 2016.

[4] Holovaty, Adrian and Kaplan-Moss, Jacob. The Definitive Guide to Django, Apress Publications.

[5] Sommerville, Ian. Software Engineering, Pearson Education, 10th Edition, 2015.

[6] Pedregosa, F. et al. Scikit-learn: Machine Learning in Python, Journal of Machine Learning Research, Volume 12, 2011.

[7] Han, Jiawei, Kamber, Micheline, and Pei, Jian. Data Mining: Concepts and Techniques, Morgan Kaufmann, 3rd Edition, 2011.

References

1. https://www.djangoproject.com
2. https://docs.djangoproject.com
3. https://www.python.org
4. https://www.sqlite.org
5. https://www.w3schools.com
6. https://developer.mozilla.org
7. https://getbootstrap.com
8. https://stackoverflow.com
9. https://www.geeksforgeeks.org
10. https://github.com
11. https://scikit-learn.org
12. https://pandas.pydata.org

APPENDIX

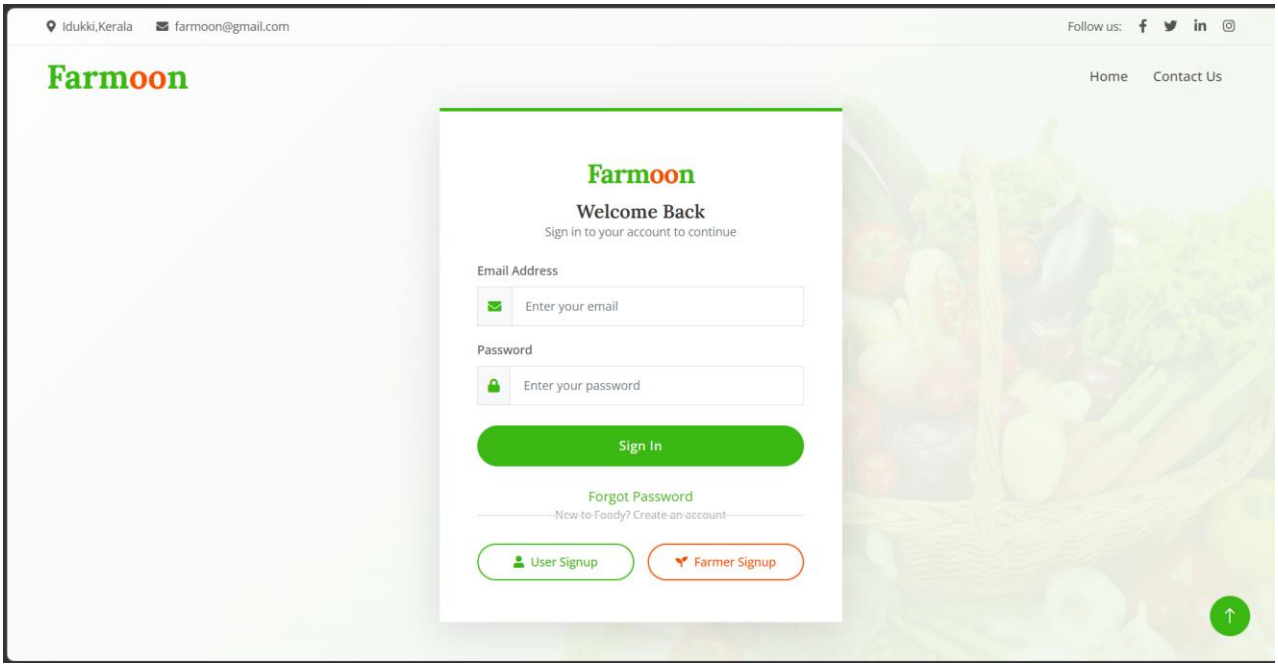
10. APPENDIX

a. Screenshots

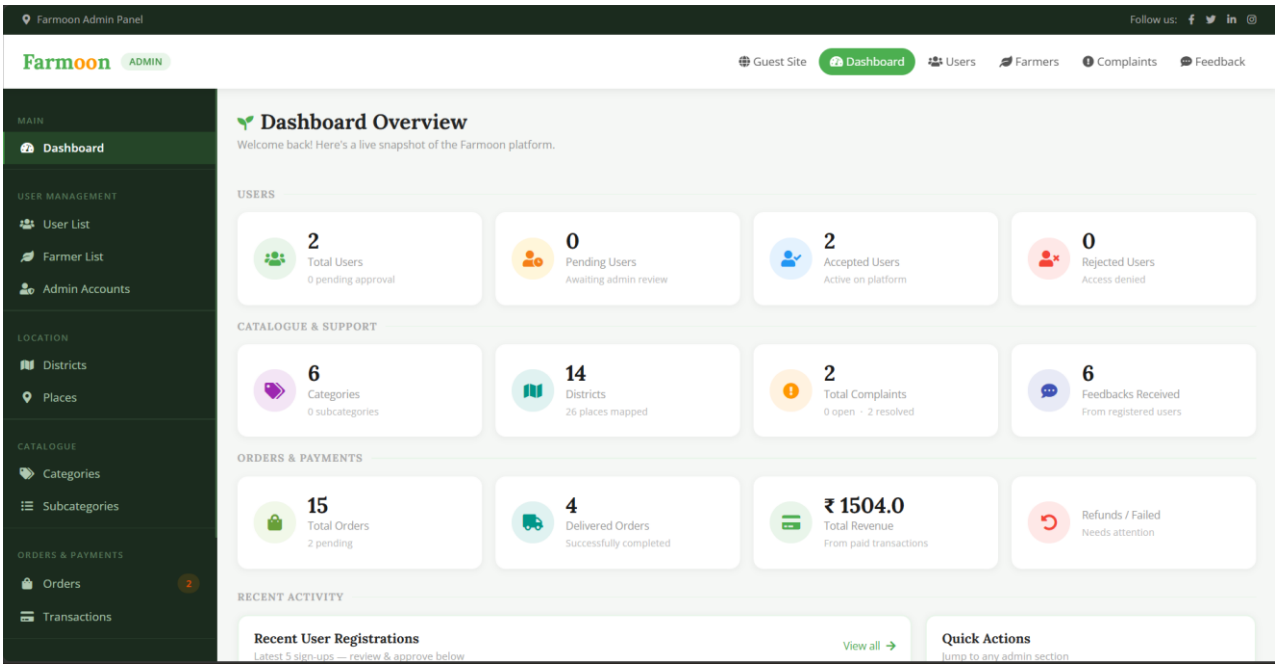
The following pages present screenshots of the major screens of FARMOON. Run the application locally (python manage.py runserver) and capture each screen, then paste the images in the marked placeholders below.

ADMIN

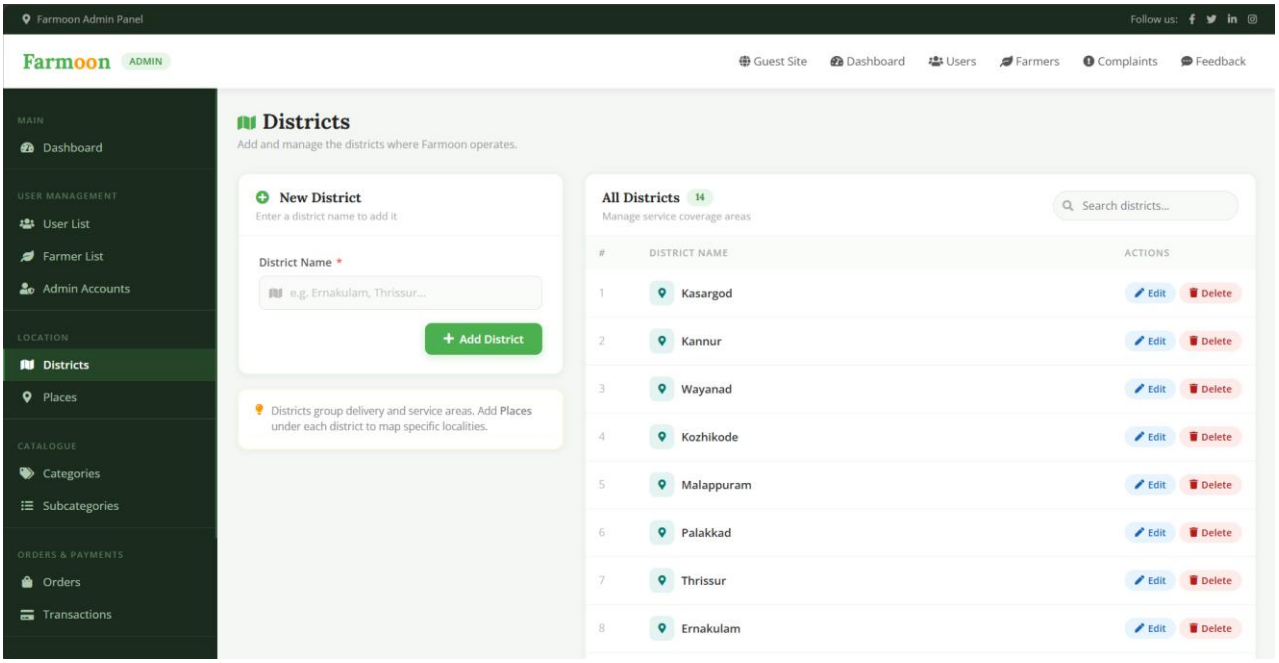
LOGIN PAGE:



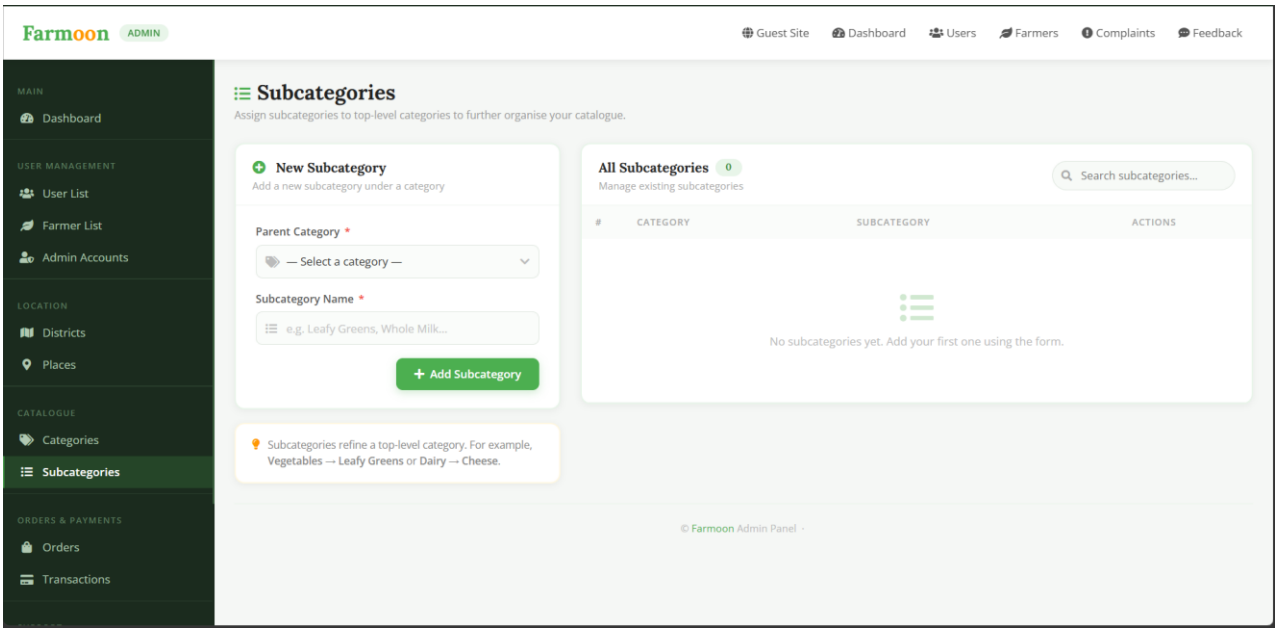
HOME PAGE:



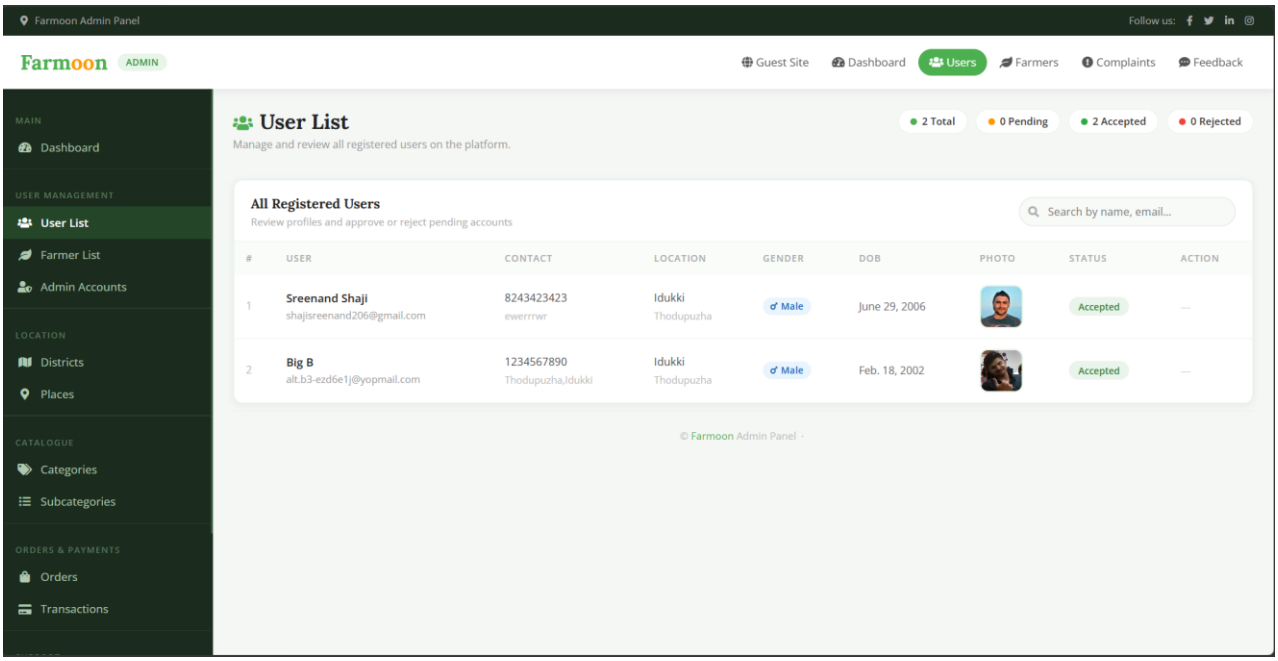
DISTRICT MANAGEMENT:



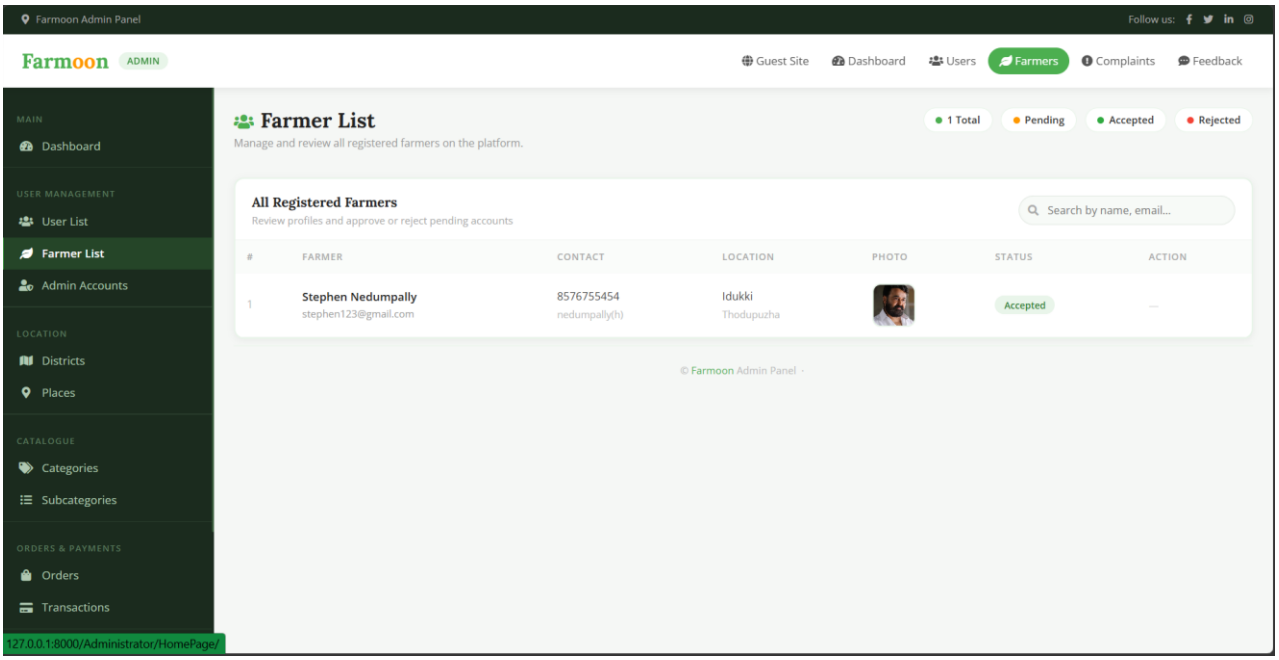
SUBCATEGORY MANAGEMENT:



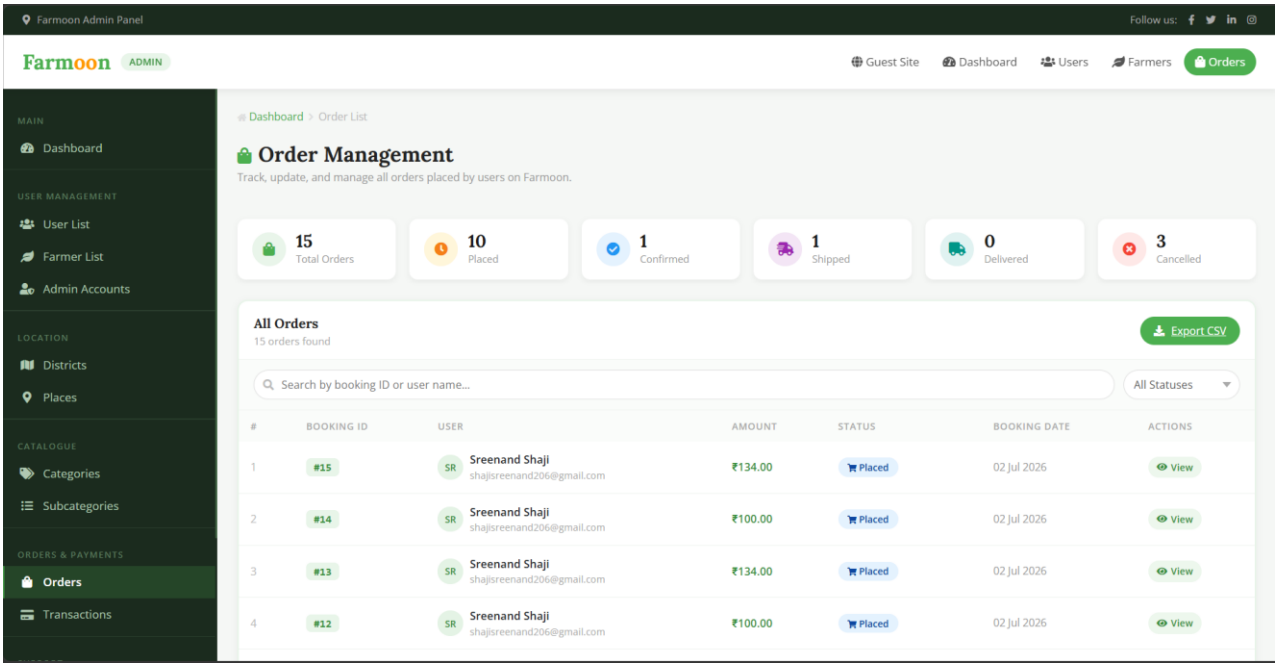
USER VERIFICATION:



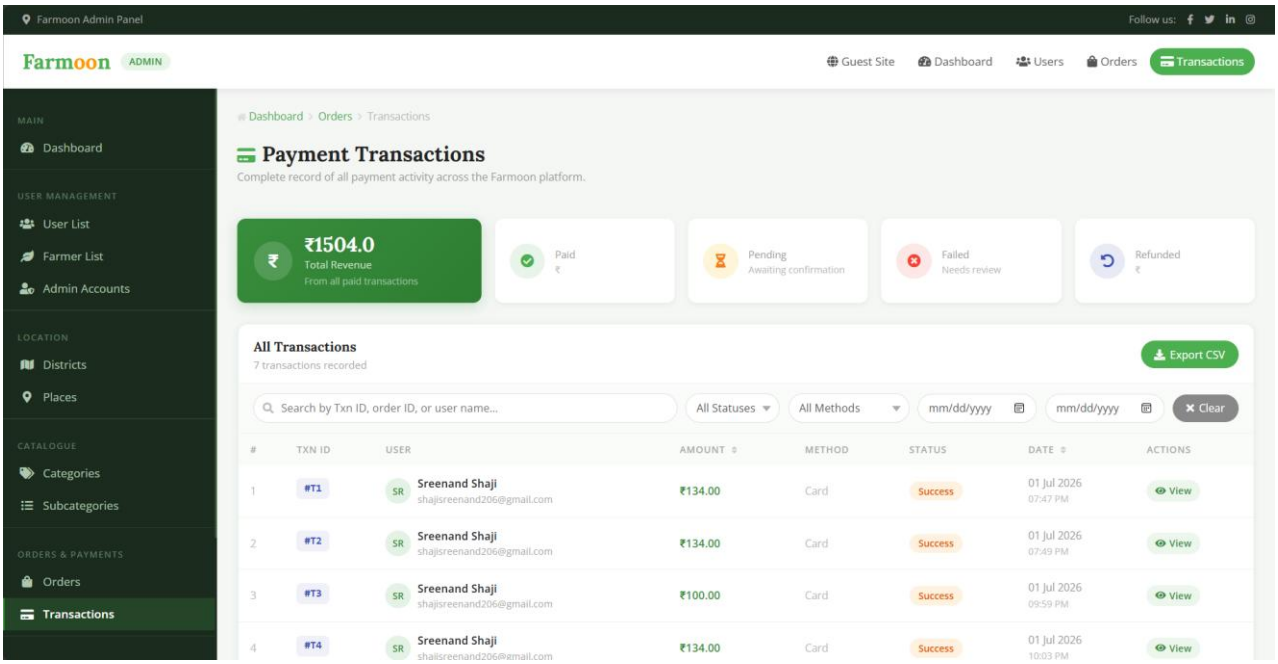
FARMER VERIFICATION:



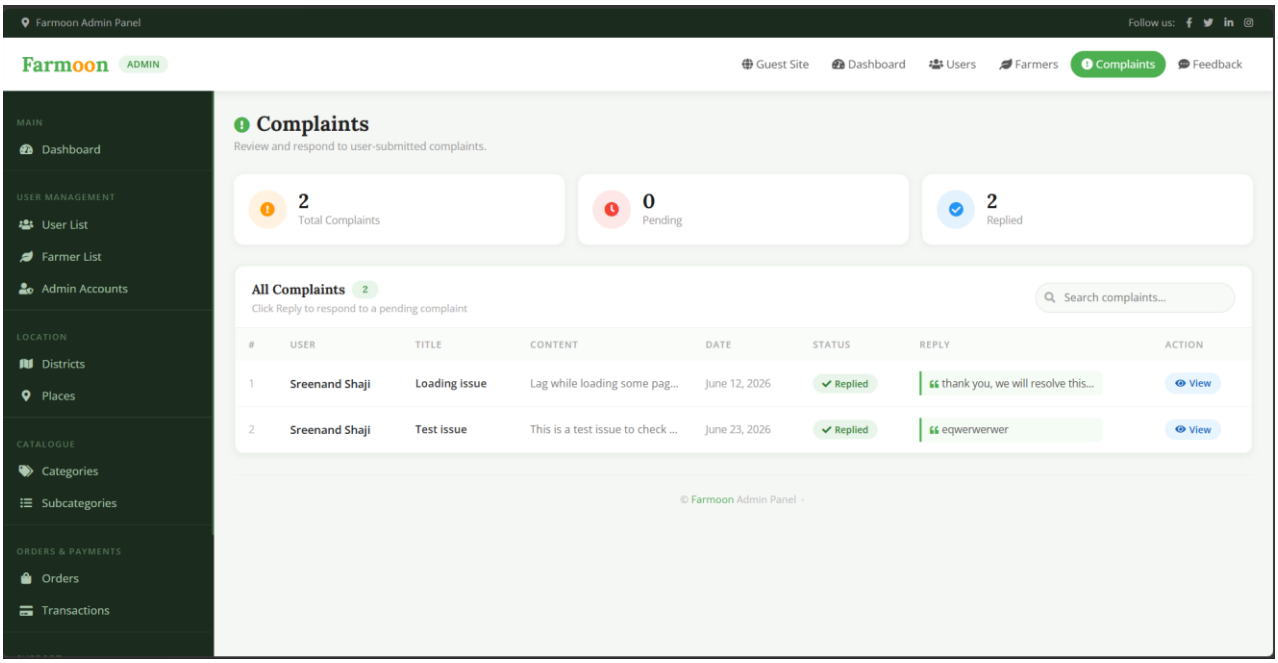
ORDER & ORDER DETAILS:



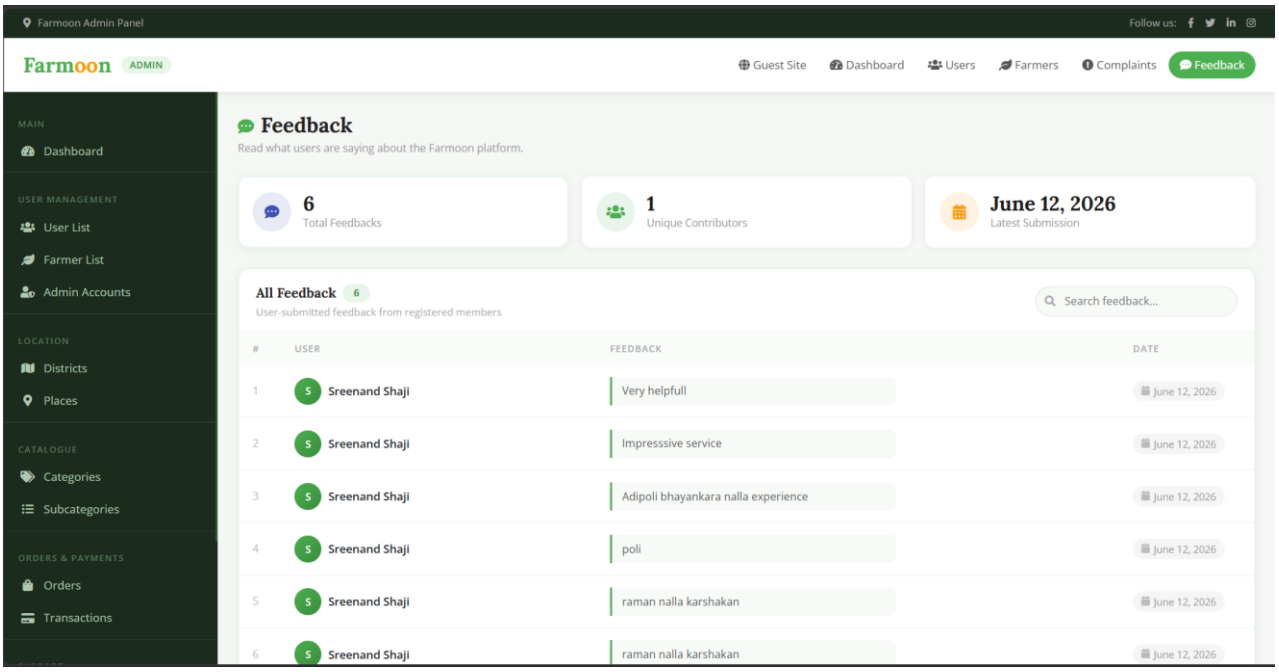
TRANSACTION LIST:



VIEW COMPLAINT:

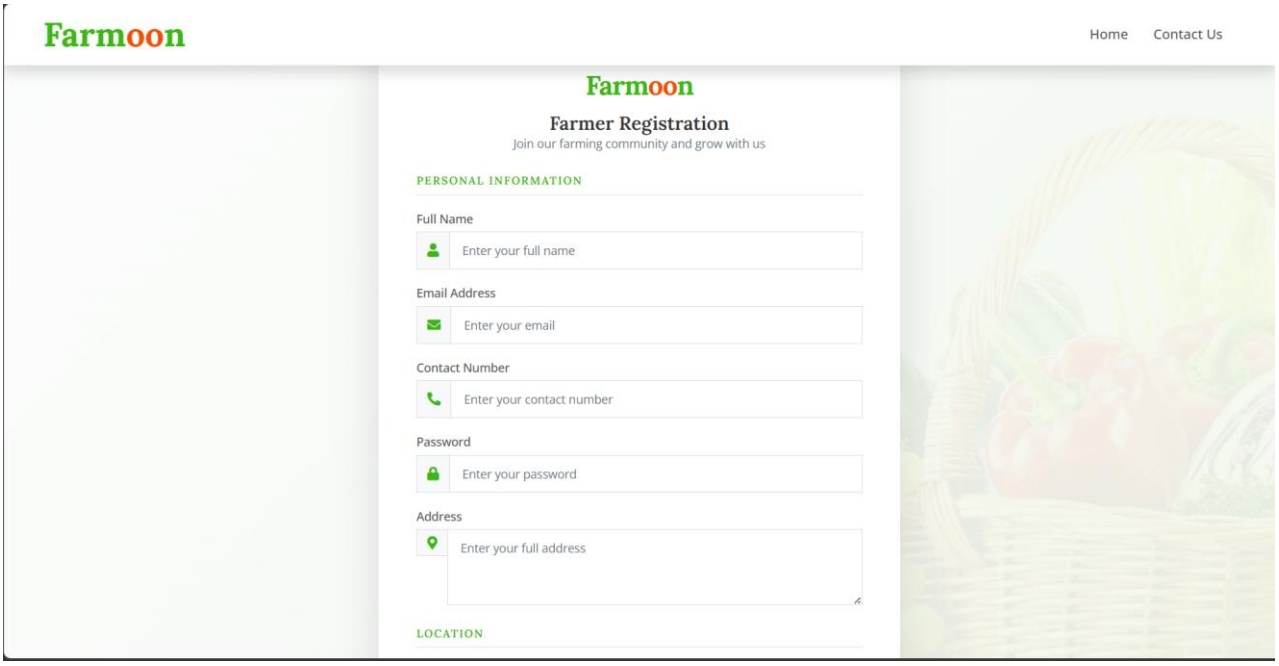


VIEW FEEDBACK:

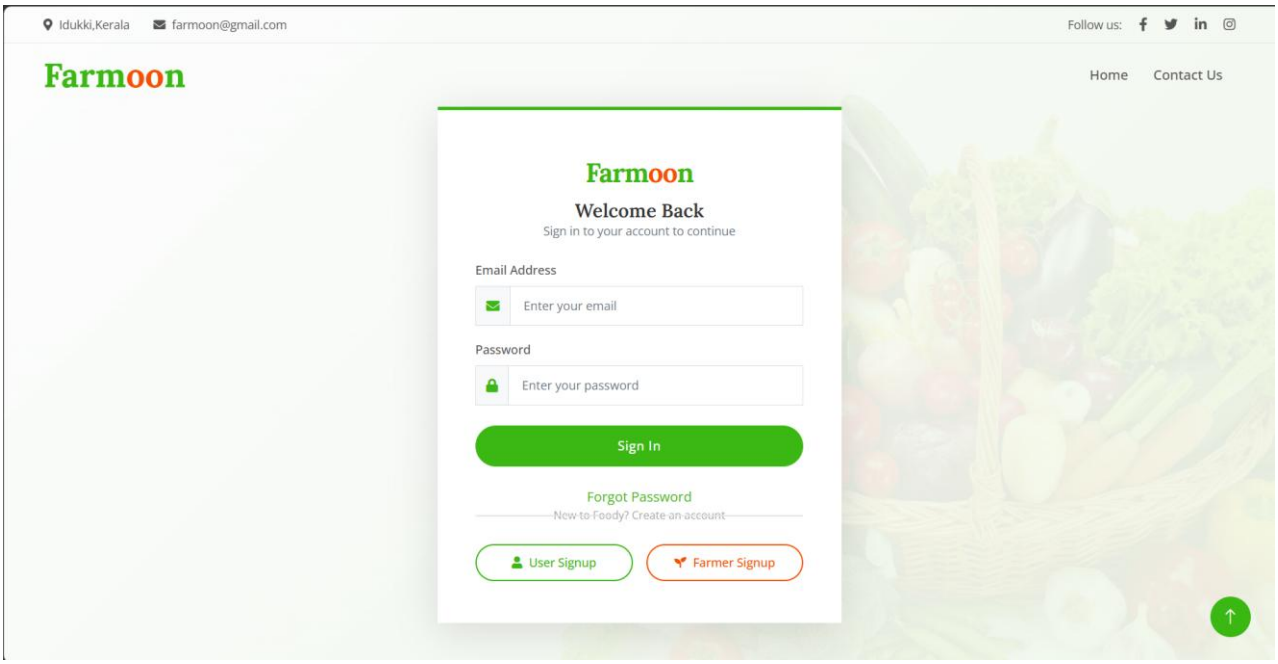


FARMER

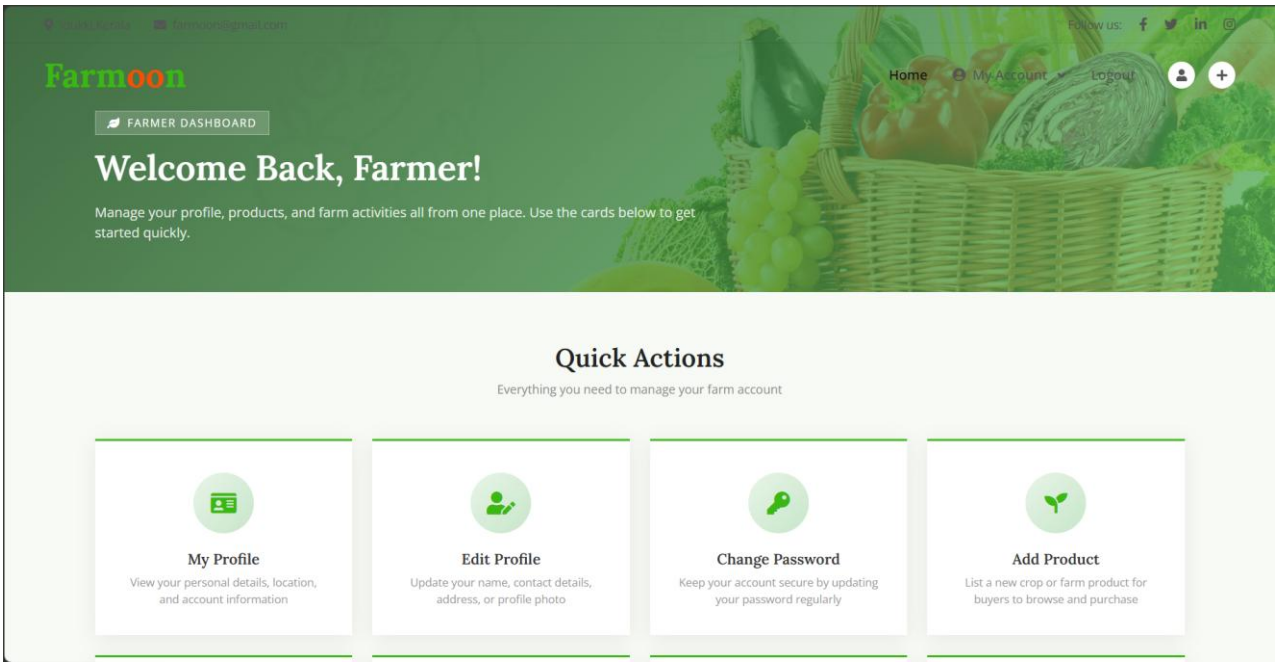
REGISTRATION PAGE:

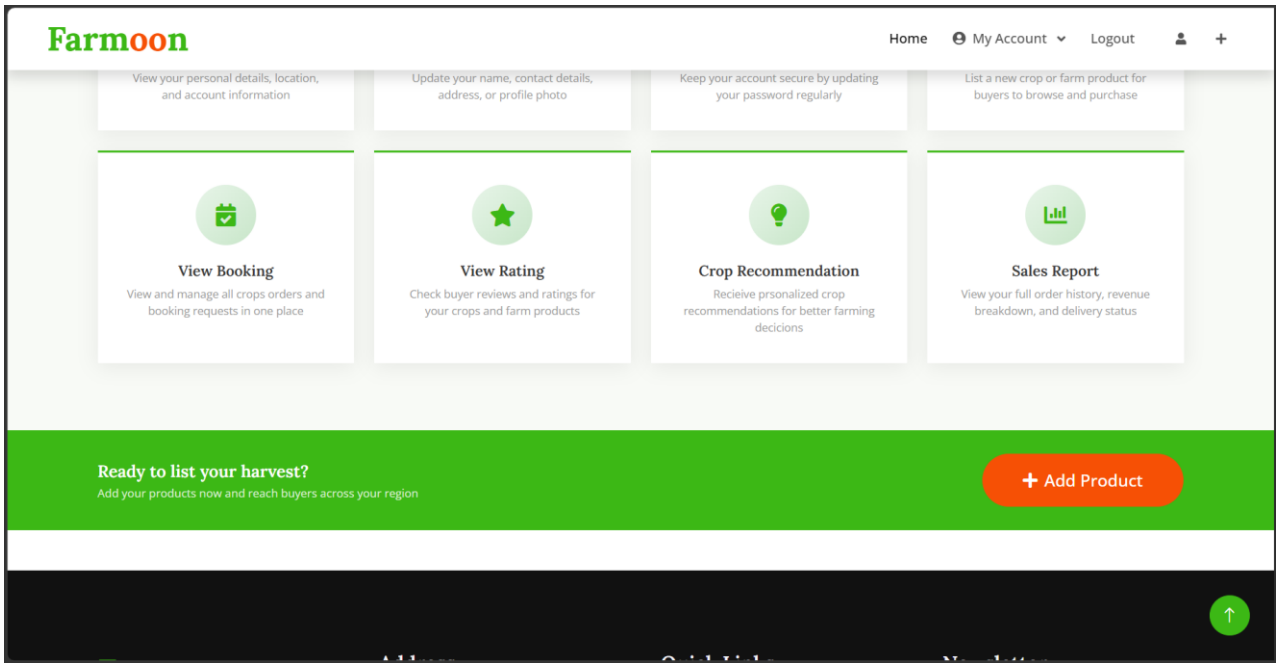


LOGIN:

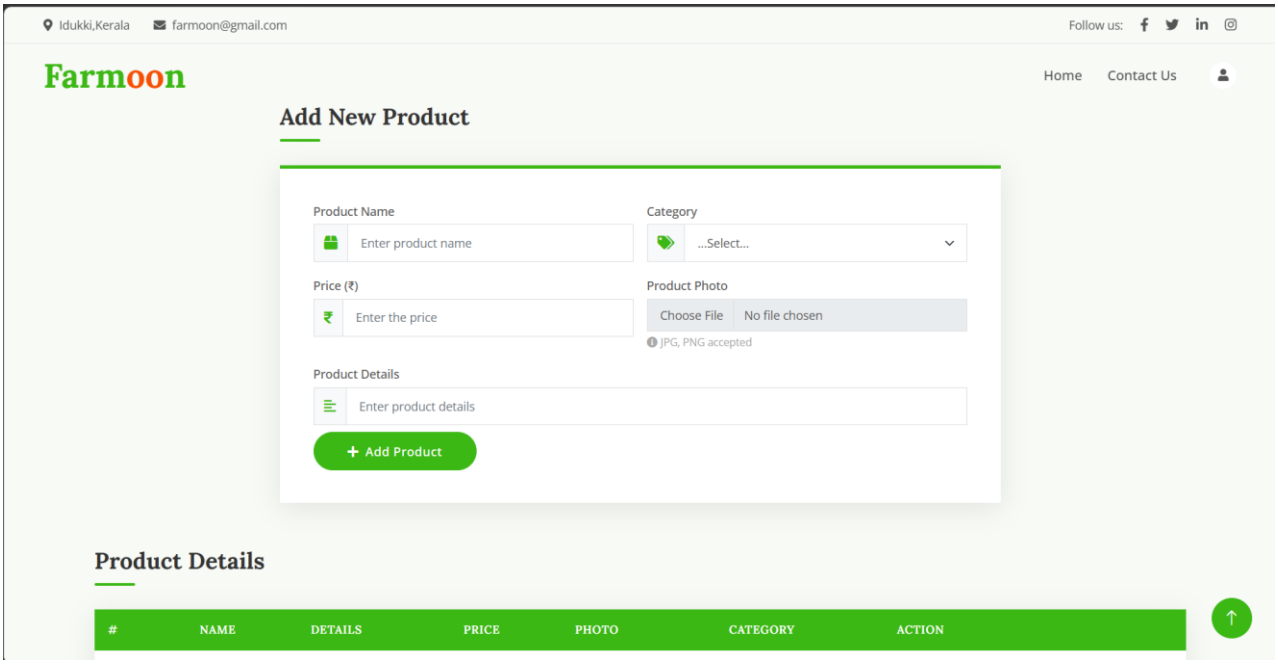


HOME PAGE/DASHBOARD:

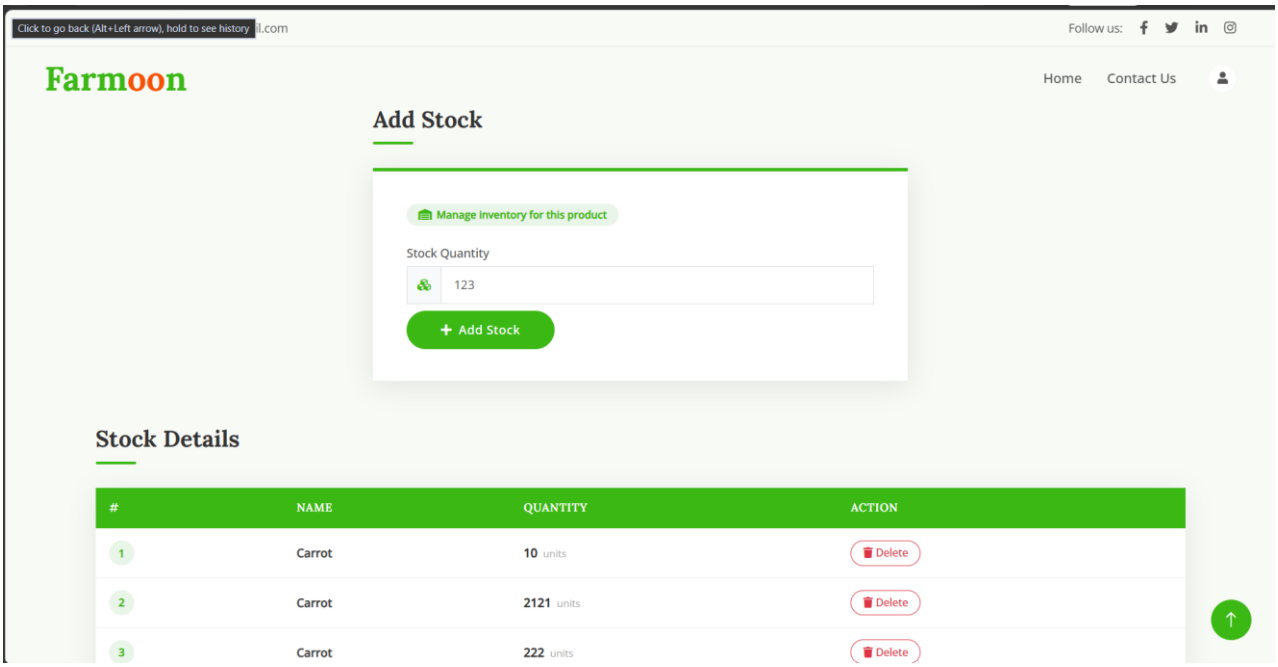



Farmoon
Home
My Account
Logout
View your personal details, location, and account information
Update your name, contact details, address, or profile photo
Keep your account secure by updating your password regularly
List a new crop or farm product for buyers to browse and purchase
View Booking
View and manage all crops orders and booking requests in one place
View Rating
Check buyer reviews and ratings for your crops and farm products
Crop Recommendation
Recieive prsonalized crop recommendations for better farming decicions
Sales Report
View your full order history, revenue breakdown, and delivery status
Ready to list your harvest?
Add your products now and reach buyers across your region
+ Add Product

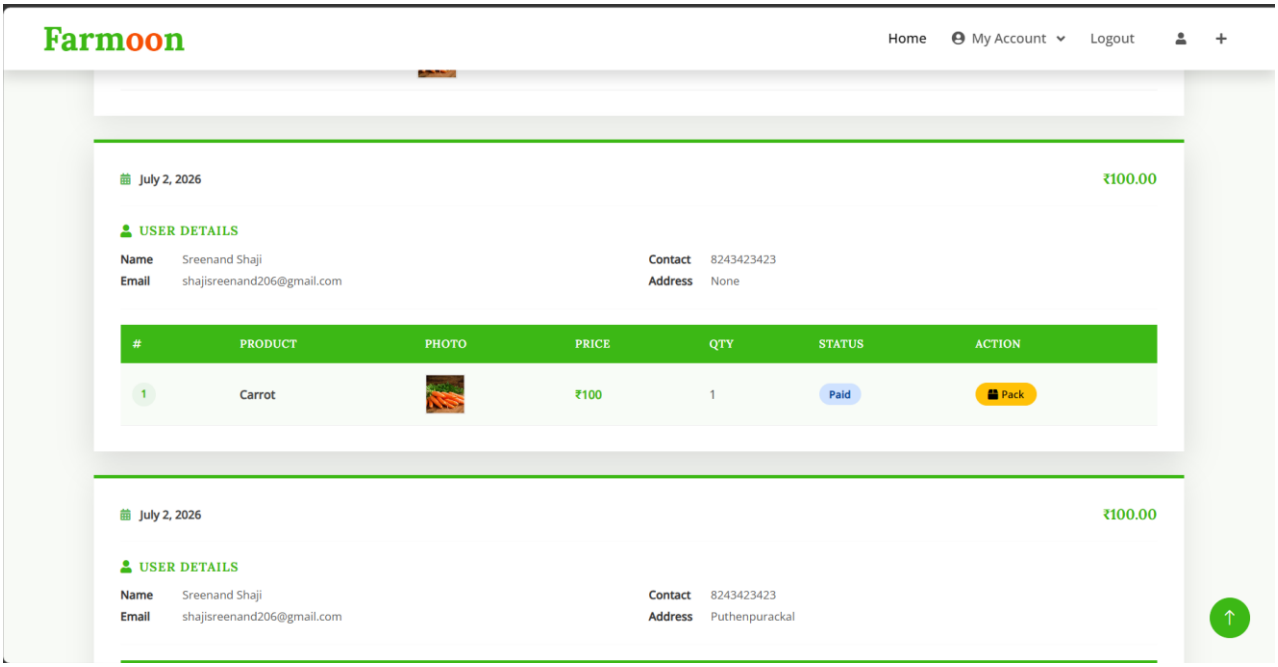
ADD PRODUCT:


Idukki,Kerala
farmoon@gmail.com
Follow us:
Farmoon
Home
Contact Us
Add New Product
Product Name
Enter product name
Category
...Select...
Price (₹)
Enter the price
Product Photo
Choose File
No file chosen
JPG, PNG accepted
Product Details
Enter product details
+ Add Product
Product Details
#
NAME
DETAILS
PRICE
PHOTO
CATEGORY
ACTION

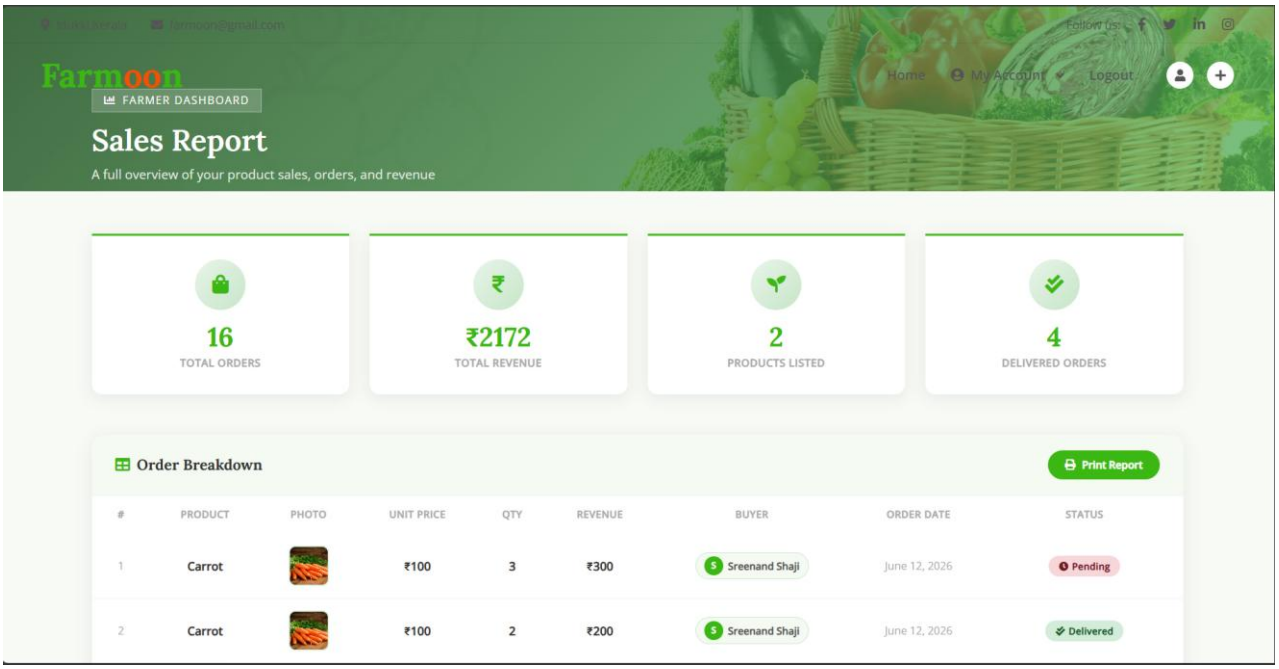
STOCK MANAGEMENT:



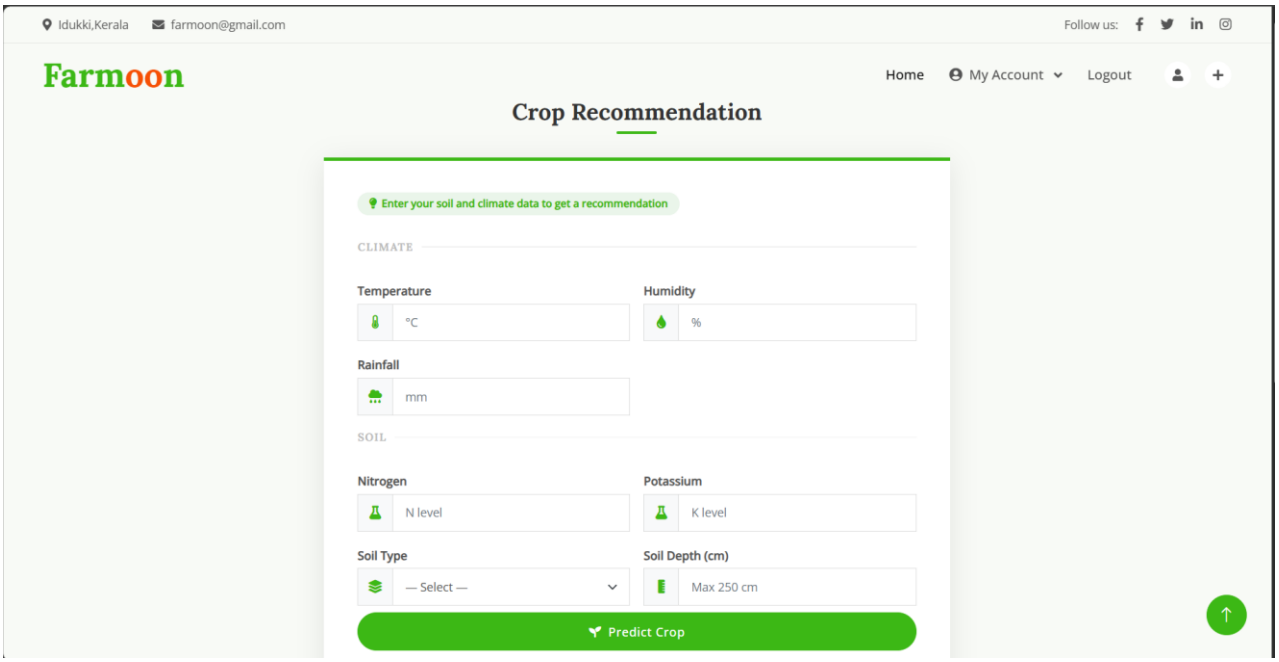
VIEW BOOKING:



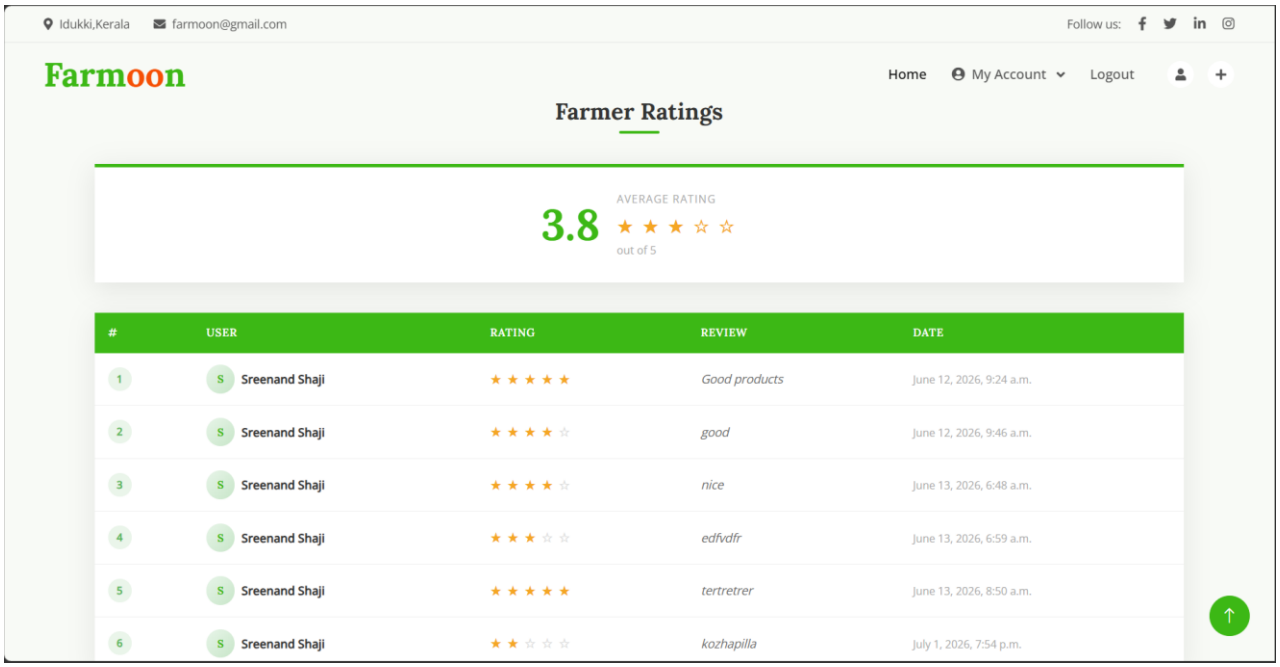
SALES REPORT:



CROP RECOMMENDATION:

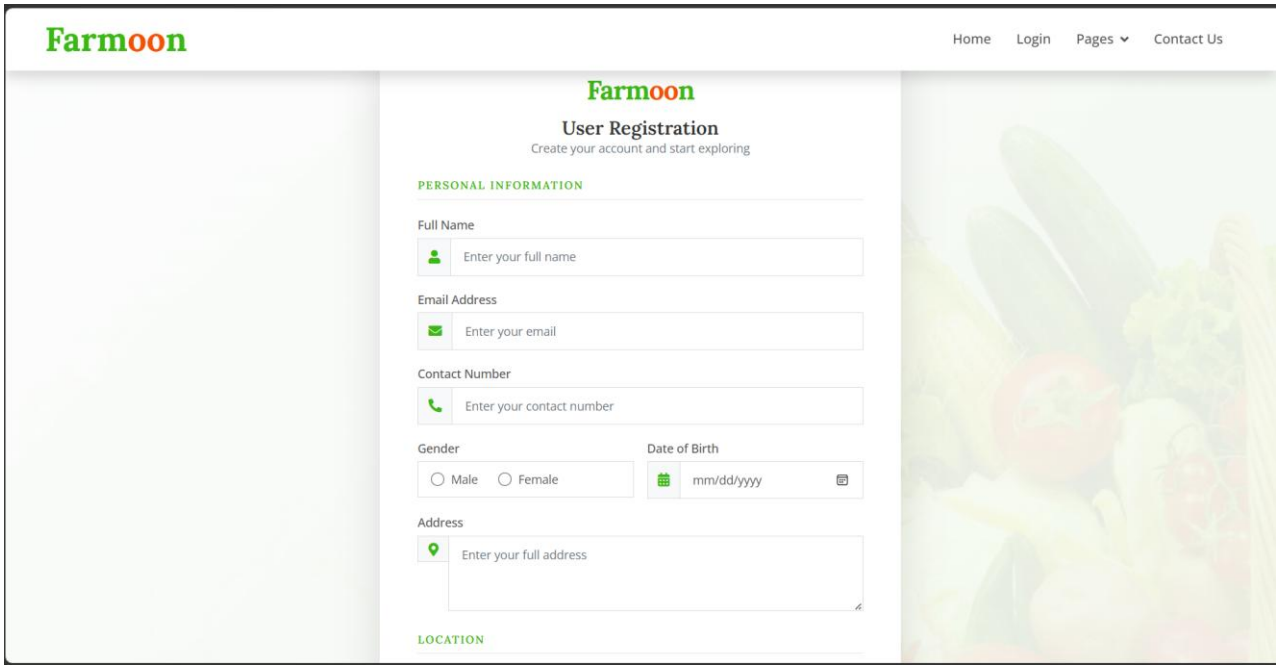


VIEW RATING:

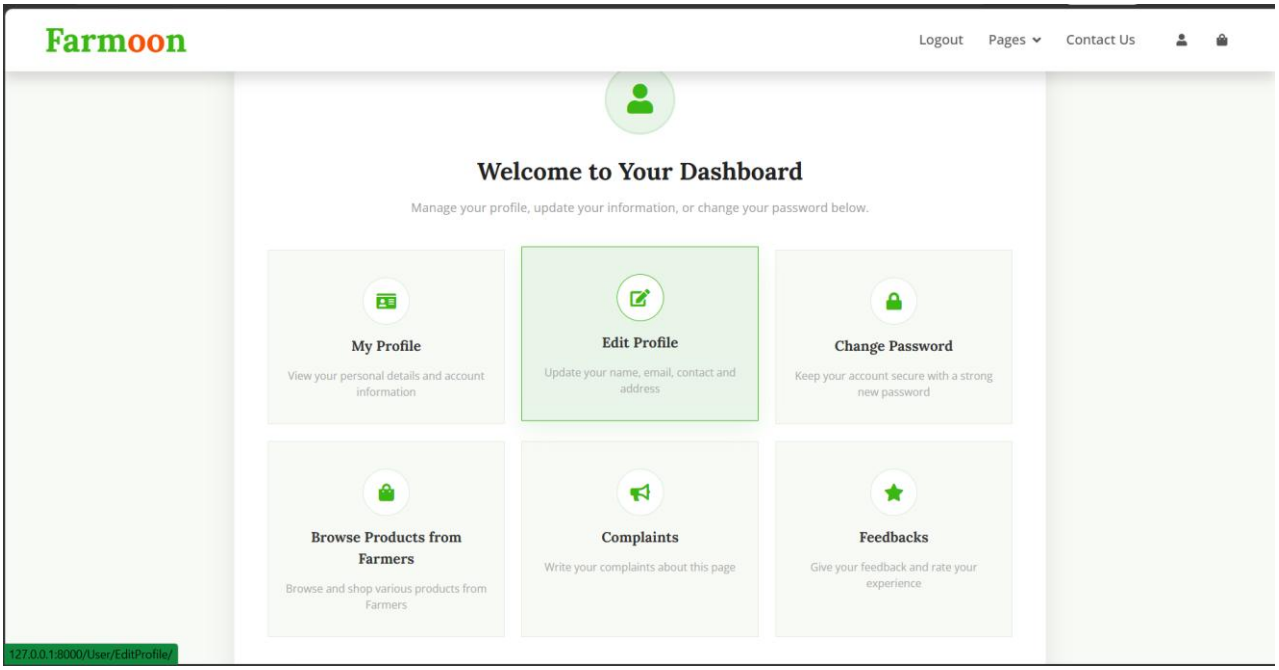


USER

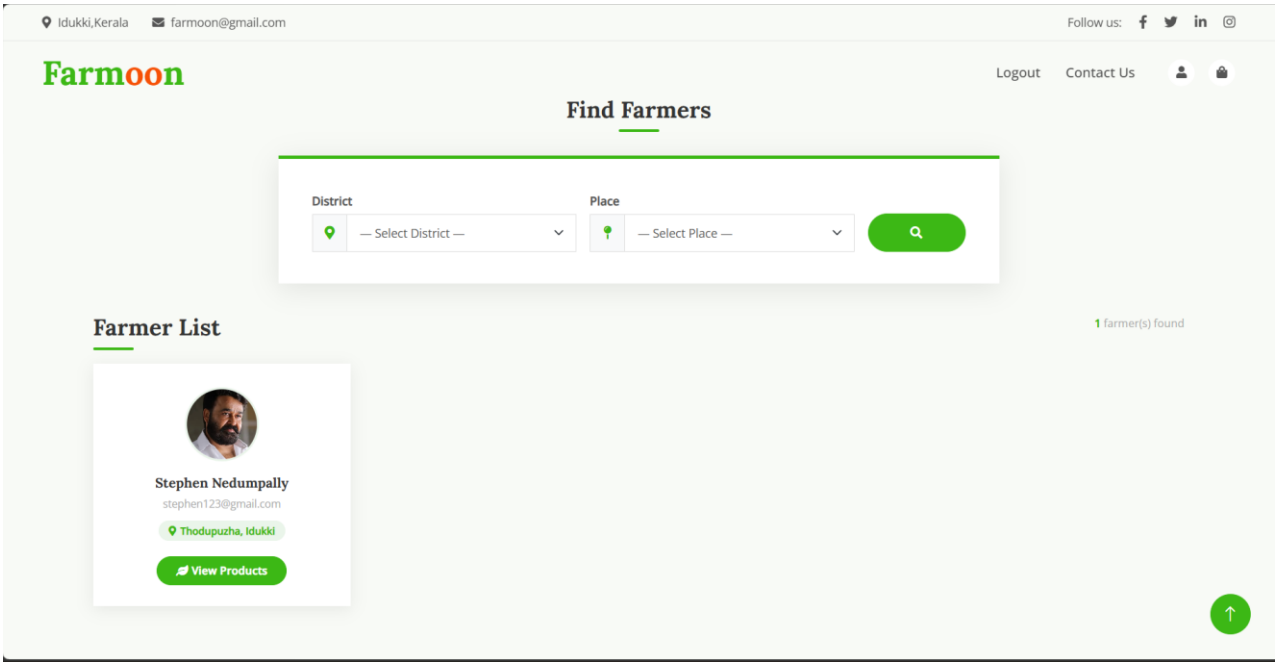
REGISTRATION PAGE:



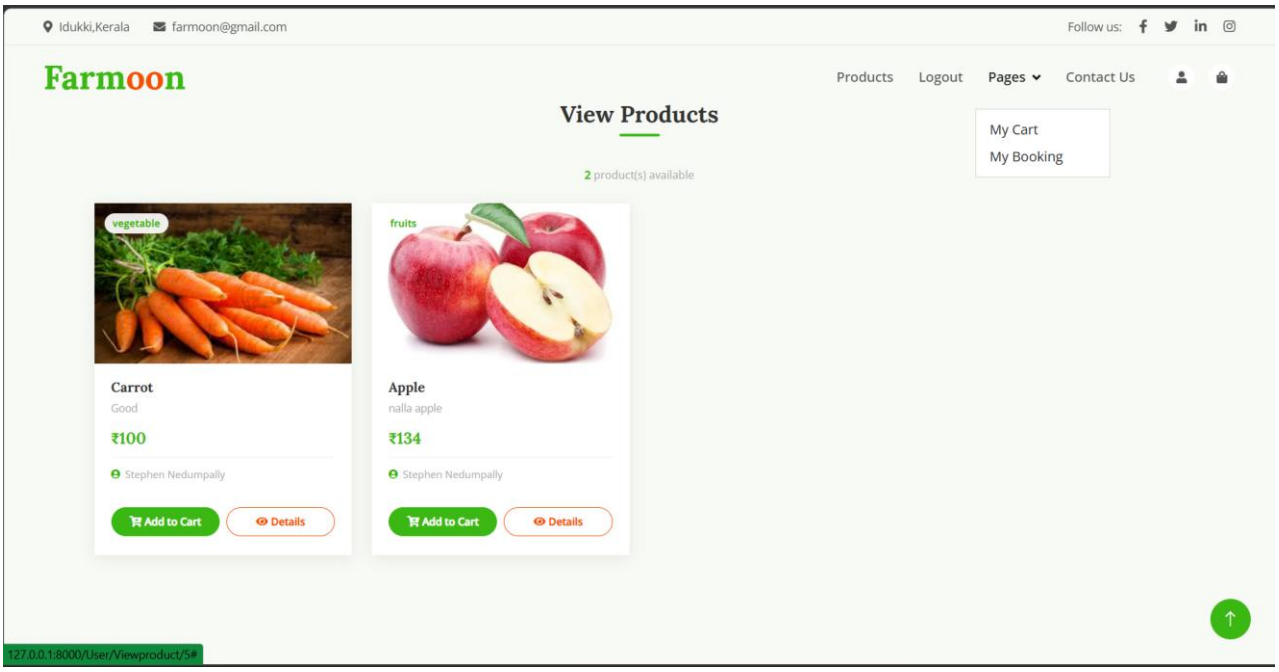
HOME PAGE:



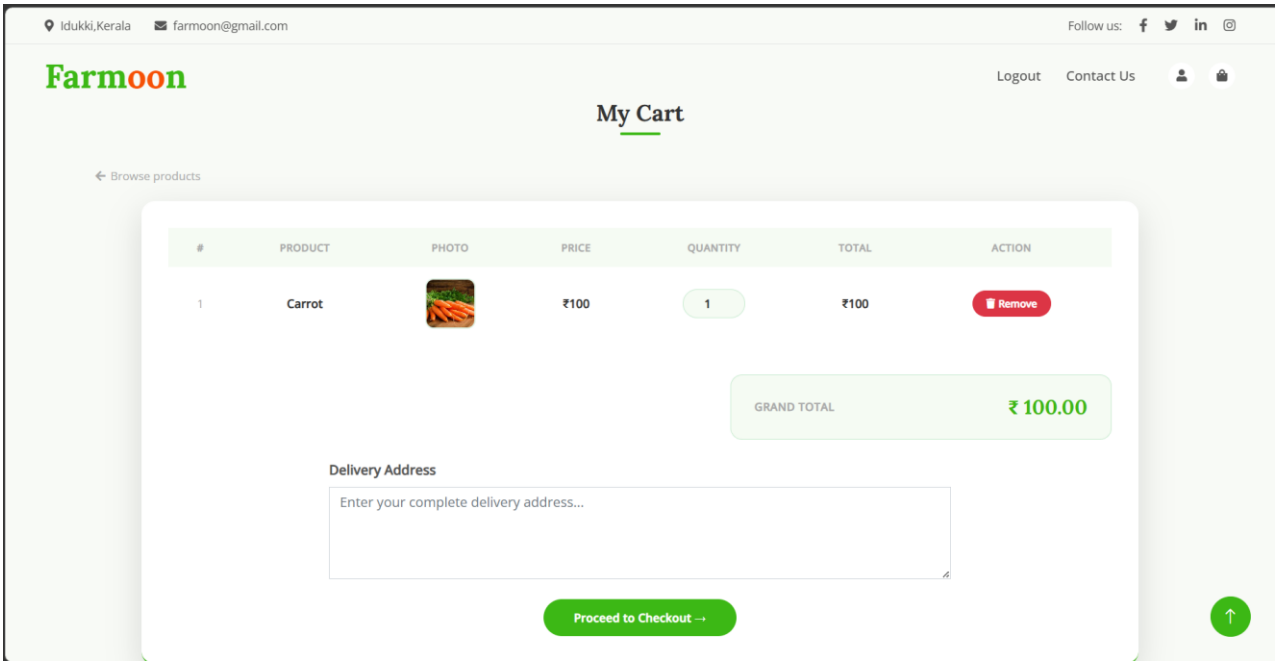
VIEW FARMERS:



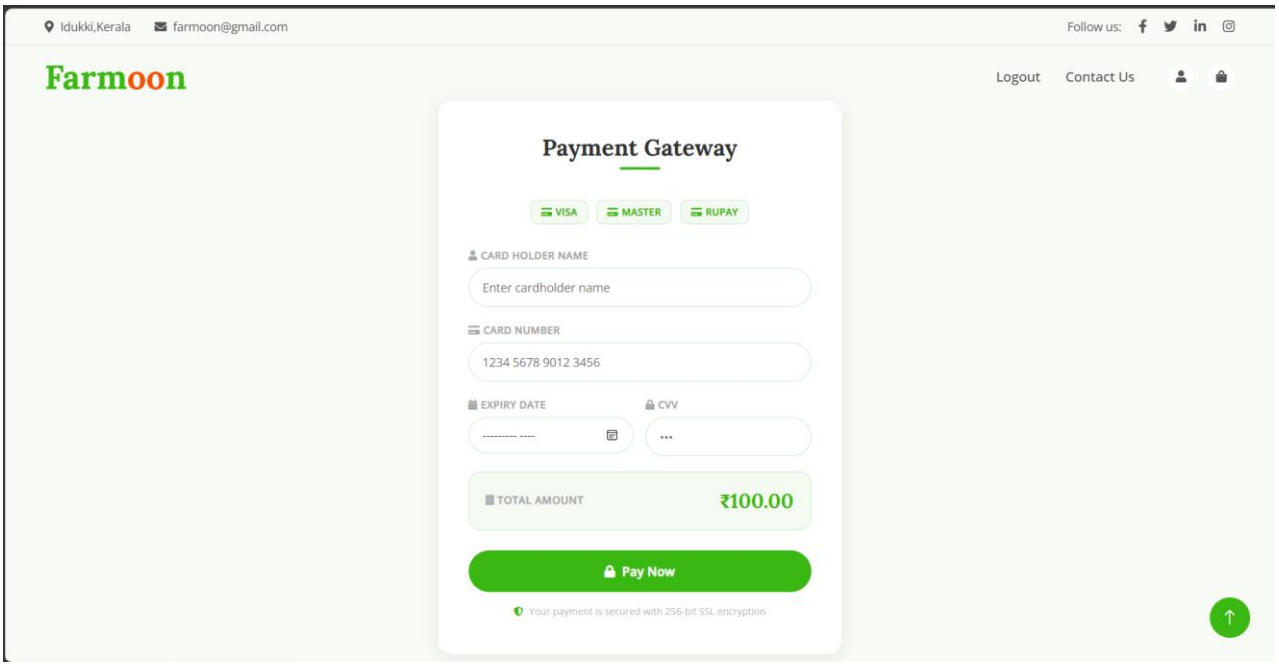
VIEW PRODUCT:



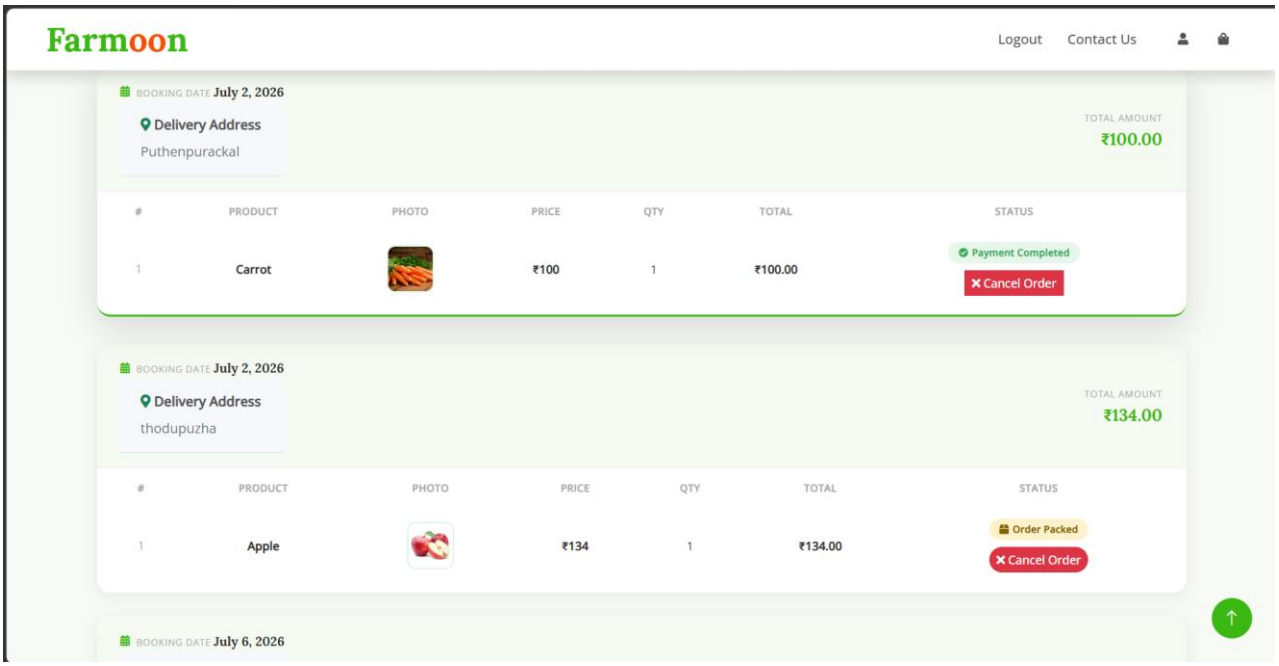
MY CART:



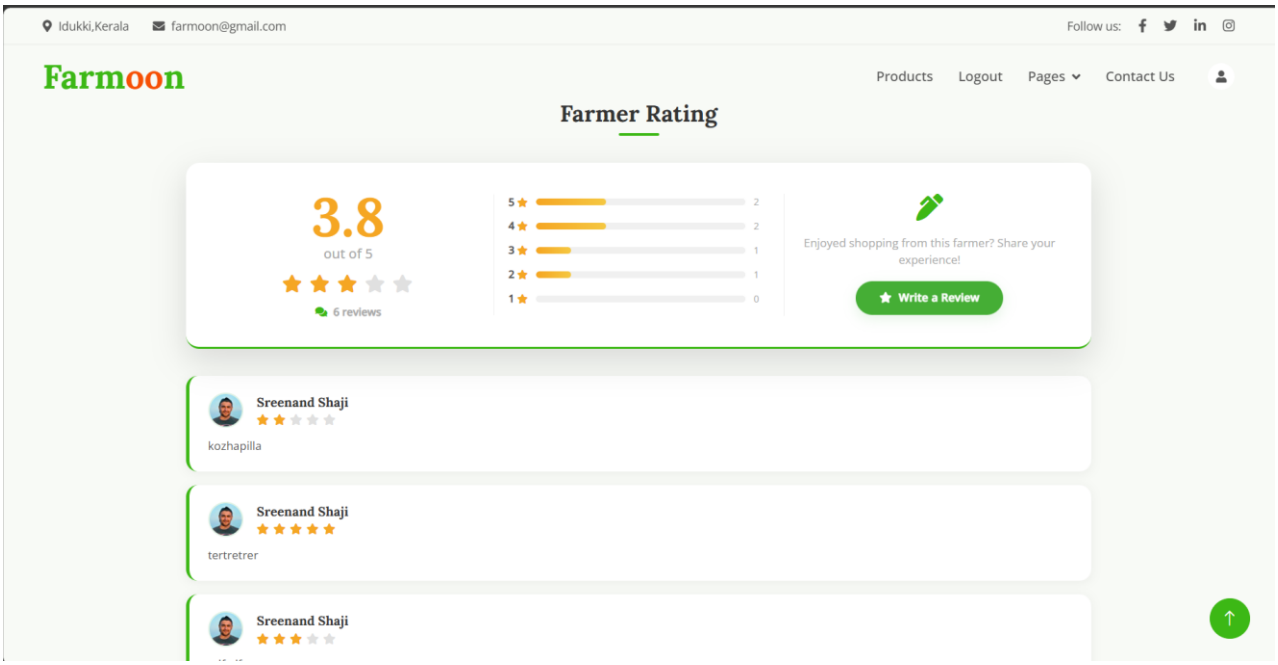
PAYMENT PAGE:



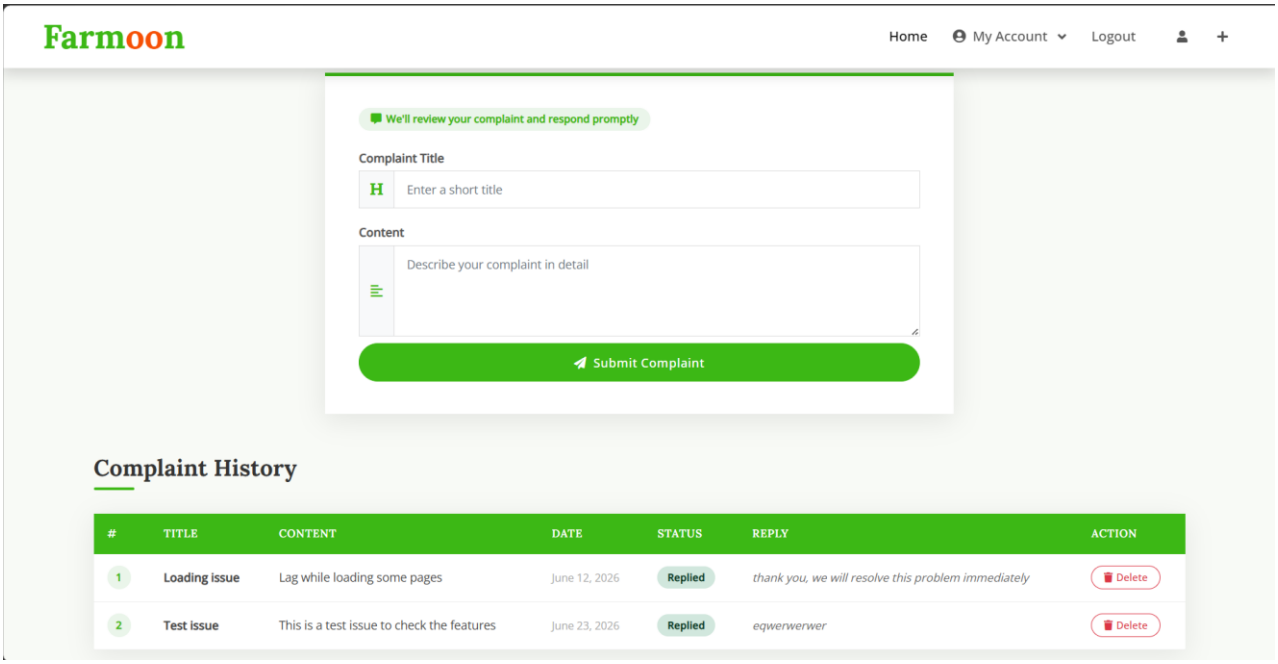
MY BOOKINGS:



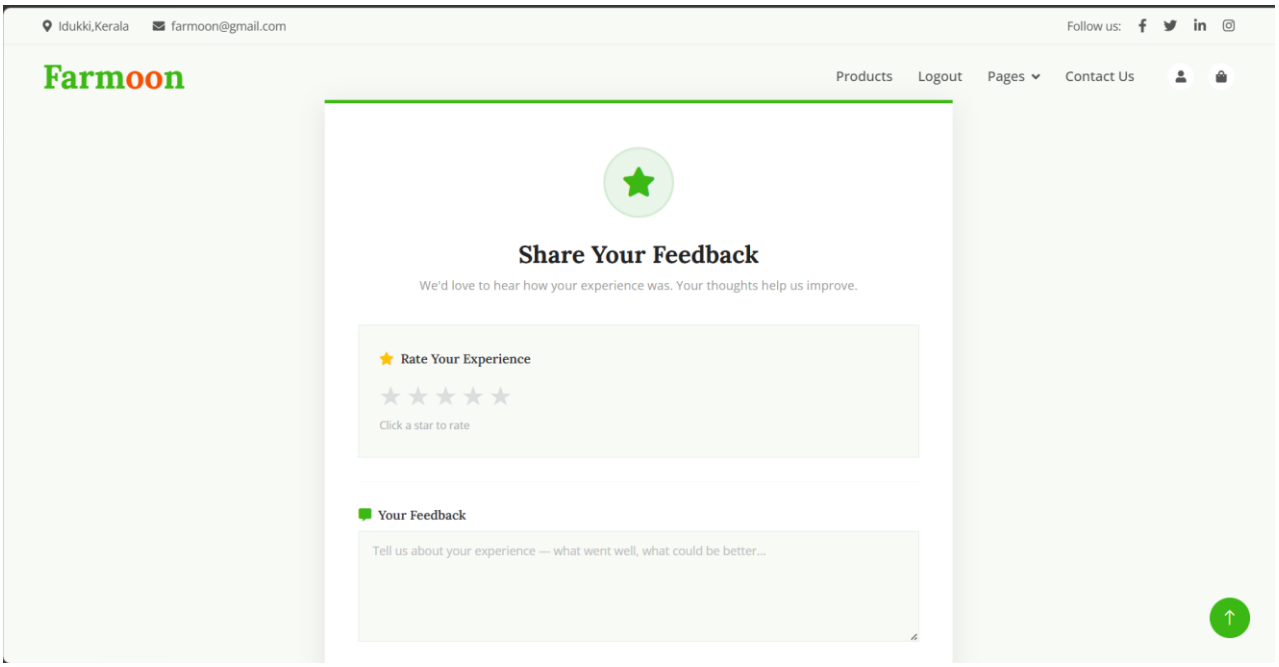
RATING PAGE:



COMPLAINT PAGE:



FEEDBACK PAGE:



b. CODING

Login (Guest/urls.py)

```
                                from django.urls import path
from . import views
app_name = 'Guest'
urlpatterns = [
    path('Login/', views.login, name='Login'),
    path('UserRegistration/', views.userregistration, name='UserRegistration'),
    path('Ajaxplace/', views.ajaxplace, name="Ajaxplace"),
    path('FarmerRegistration/', views.farmerregistration, name="FarmerRegistration"),
    path('', views.index, name='index'),
    path('AboutUs/', views.aboutus, name='AboutUs'),
path('ContactUs/', views.contactus, name='ContactUs'),
path('Product/', views.product, name='Product'),
    path('OTP/', views.otp, name='OTP'),
    path('ForgotPassword/', views.forgotpassword, name='ForgotPassword'),
    path('NewPassword/', views.newpassword, name='NewPassword'),
]
```

Login View (Guest/views.py)

```
def login(request):
    if request.method == "POST":
        email = request.POST.get('txt_Email')
password = request.POST.get('txt_password')
usercount = tbl_userregistration.objects.filter(
            userregistration_email=email, userregistration_password=password
            ).count()
        admincount = tbl_adminregistration.objects.filter(
            adminregistration_email=email, adminregistration_password=password
            ).count()
        farmercount = tbl_farmerregistration.objects.filter(
farmer_email=email, farmer_password=password
).count()         if usercount > 0:             userdata =
tbl_userregistration.objects.get(
                userregistration_email=email, userregistration_password=password
)             if userdata.user_status == 0:
                return render(request, 'Guest/Login.html', {"Msg": "Registration Pending"})
elif userdata.user_status == 2:                 return render(request, 'Guest/Login.html',
{"Msg": "Registration Rejected"})             else:                 request.session["uid"] =
userdata.id                 return redirect("User:HomePage")         elif admincount > 0:
admindata = tbl_adminregistration.objects.get(
                adminregistration_email=email, adminregistration_password=password
                )
            request.session["aid"] = admindata.id
return redirect("Administrator:HomePage")         elif
farmercount > 0:             farmerdata =
tbl_farmerregistration.objects.get(
farmer_email=email, farmer_password=password
                )
            request.session["fid"] = farmerdata.id             return
redirect("Farmer:HomePage")         else:             return render(request,
'Guest/Login.html', {"Msg": "Invalid Email or Password"})     else:
```

```
        return render(request, 'Guest/Login.html')
```

User Registration View (Guest/views.py)

```
def userregistration(request):     userregistration_data =
tbl_userregistration.objects.all()
    district_data = tbl_district.objects.all()     place_data =
tbl_place.objects.all()     if request.method == "POST":
name = request.POST.get('txt_name')         email =
request.POST.get('txt_Email')         contact =
request.POST.get('num_contact')         address =
request.POST.get('txt_Address')         gender =
request.POST.get('rd_Gender')         dateofbirth =
request.POST.get('txt_date')         district_id =
request.POST.get("sel_district")         district_obj =
tbl_district.objects.get(id=district_id)
        place_id = request.POST.get("txt_place")
place_obj = tbl_place.objects.get(id=place_id)
photo = request.FILES.get('file_photo')
password = request.POST.get('txt_password')
tbl_userregistration.objects.create(
            userregistration_name=name, userregistration_email=email,
userregistration_contact=contact, userregistration_address=address,
userregistration_gender=gender, userregistration_dateofbirth=dateofbirth,
            userregistration_place=place_obj, userregistration_photo=photo,
            userregistration_password=password)
return redirect("Guest:UserRegistration")     else:
return render(request, 'Guest/UserRegistration.html',
                      {'userregistration': userregistration_data,
                       'district': district_data, 'place': place_data})
```

Feedback View (User/views.py)

```
def feedback(request):     feedbackdata =
tbl_feedback.objects.filter(user=request.session['uid'])     if
request.method == 'POST':
        content = request.POST.get("txt_feedback")
rating = request.POST.get("txt_rating")
        tbl_feedback.objects.create(
feedback_content=content,
feedback_rating=int(rating),
            user=tbl_userregistration.objects.get(id=request.session['uid'])
        )
        return redirect("User:HomePage")     else:         return
render(request, "User/Feedback.html", {'feedback': feedbackdata})
```

View Feedback (Administrator/views.py)

```
def viewfeedback(request):     data =
tbl_feedback.objects.all()
    unique_users_count = data.values('user_id').distinct().count()
    latest = data.order_by('-feedback_date').first()
    latest_submission = latest.feedback_date if latest else None

    return render(request, 'Administrator/ViewFeedback.html', {
        'data': data,
        'unique_users_count': unique_users_count,
        'latest_submission': latest_submission,
    })
```

Add Product View (Farmer/views.py)

The Add Product module allows a verified farmer to list a new product for sale, associating it with a category and storing the product photo, price, and description in the tbl_product table.

Machine Learning Crop Recommendation View (Farmer/views.py)

Soil Type	Encoded Value
Alluvial	0
Black	1
Clayey	2
Loamy	3
Sandy	4

This view loads the pre-trained KNN classification model (knn_model.pkl) once using Joblib, then, on each form submission, builds a single-row Pandas DataFrame from the farmer's soil and climate inputs and passes it to the model to obtain a predicted crop, which is displayed back to the farmer.

```
import os import
joblib import
pandas as pd


# Define the correct model path
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
MODEL_PATH = os.path.join(BASE_DIR, 'Assets', 'Model', 'knn_model.pkl')


# Load the trained K-Nearest Neighbors (KNN) model
model = joblib.load(MODEL_PATH)
def croprecommend(request):
if request.method == 'POST':
        # Get input values from the form
nitrogen = float(request.POST['nitrogen'])
potassium = float(request.POST['potassium'])
temperature = float(request.POST['temperature'])
humidity = float(request.POST['humidity'])
rainfall = float(request.POST['rainfall'])
soiltype = int(request.POST['soiltype'])
soil_depth = float(request.POST['soil_depth'])

        # Create DataFrame for model prediction
        input_data = pd.DataFrame({
            'N': [nitrogen],
            'K': [potassium],
            'temperature': [temperature],
            'humidity': [humidity],
            'rainfall': [rainfall],
            'soiltype': [soiltype],
            'Soil_Depth_cm': [soil_depth]
        })

        # Make prediction
        prediction = model.predict(input_data)
recommended_crop = prediction[0]

        return render(request, 'Farmer/CropRecommendation.html', {'res': recommended_crop})

    return render(request, 'Farmer/CropRecommendation.html')
```

The Soil Type field is presented to the farmer as a dropdown and is encoded numerically before being sent to the model, as follows:

Booking / Cart Model (User/models.py)

```
class tbl_booking(models.Model):     booking_date =
models.DateField(auto_now_add=True)     booking_time =
models.TimeField(auto_now_add=True)     booking_status
= models.IntegerField(default=0)     booking_amount =
models.CharField(max_length=60)
    userid = models.ForeignKey(tbl_userregistration, on_delete=models.CASCADE, null=True)
    delivery_address = models.TextField(null=True, blank=True)
class tbl_cart(models.Model):     cart_quantity =
models.IntegerField(default=1)     cart_status =
models.IntegerField(default=0)
    productid = models.ForeignKey(tbl_product, on_delete=models.CASCADE, null=True)
bookingid = models.ForeignKey(tbl_booking, on_delete=models.CASCADE, null=True)
```